Università di Genova
DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

HookDesigner: A Code Generator for Hook-Based Dynamic Binary Instrumentation on Windows

Davide Giusto

Master's Thesis

Università di Genova, DIBRIS
Via Dodecaneso, 35 16146 Genova, Italy
https://www.dibris.unige.it/



Computer Science MSc
Software Security and Engineering Curriculum

HookDesigner: A Code Generator for Hook-Based Dynamic Binary Instrumentation on Windows

Davide Giusto

Advisor: Giovanni Lagorio

Examiner: Marina Ribaudo

November, 2026

Abstract

This thesis presents the design and implementation of HookDesigner, a code generator that produces hook-based dynamic binary instrumentation code for Windows from a policy written in a domain-specific language (DSL). The thesis evaluates freely available Windows hooking libraries and selects some of them as HookDesigner compilation targets. The generated hooks support performance measurement, security policy enforcement, and runtime monitoring. Particular attention is given to thread safety and to reliable attachment and detachment from running processes in production settings.

Table of Contents

Chapter 1 Introduction **8**

1.1 Motivation 8

1.2 Research Question and Running Example 10

1.3 Content of the Thesis 11

Chapter 2 Background **12**

2.1 Dynamic-Link Libraries 12

2.1.1 Address Space Isolation 13

2.1.2 DLL Injection 13

2.1.3 Modules and Function Addresses 14

2.2 The Application Binary Interface (ABI) 15

2.2.1 Windows x86 (32-bit) Architecture 16

2.2.2 Windows x64 (64-bit) Architecture 17

2.3 Inline Hooking 17

2.3.1 Challenges in Inline Hooking 19

2.4 Code-Cache-Based DBI 20

2.5 Logging and Metrics 21

2.5.1 Logging with Spdlog 21

2.5.2 Metrics with Prometheus 22

2.6 Summary 23

Chapter 3 State of the Art **25**

3.1 Microsoft Detours and MinHook . . . 25

3.2 Frida-Gum . . . 27

3.2.1 Function Replacement Mode . . . 27

3.3 Hook Execution Flow . . . 28

3.3.1 Advantages of Engine-Mediated Dispatch . . . 28

3.4 Instruction Boundaries and Disassemblers . . . 30

3.5 Patching a Running Process . . . 31

3.6 Summary . . . 33

Chapter 4 From Policy to Instrumentation DLL **35**

4.1 Design Principles and Architecture . . . 35

4.1.1 Design Principles . . . 36

4.1.2 The Seven-Stage Pipeline . . . 36

4.1.3 Selection of Compilation Targets . . . 37

4.2 Declarative Policy Specification and Logic File Integration . . . 38

4.2.1 The Core YAML Policy DSL . . . 38

4.2.2 Logic File Integration . . . 42

4.3 C++ Code Generation and DLL Compilation . . . 46

4.3.1 Engine Registry . . . 47

4.3.2 Jinja2 Template Architecture . . . 48

4.3.3 Hook Function Generation . . . 50

4.3.4 Hook Lifecycle Function Generation . . . 52

4.4 HookDesigner Telemetry . . . 55

4.4.1 Telemetry Policy Specification . . . 55

4.4.2 Telemetry Expression Translation . . . 61

4.4.3 Asynchronous Logging . . . 65

4.4.4 Performance Metrics Engine . . . 67

4.5 Summary . . . 70

Chapter 5 Runtime Safety and DLL Lifecycle 72

5.1 DLL Runtime Safety . . . 72

5.1.1 Transpiler Type Analysis and Memory Access Helpers . . . 73

5.1.2 Re-entrancy Guard . . . 76

5.1.3 System Error State Preservation . . . 78

5.1.4 Asynchronous Exception Handling . . . 80

5.1.5 The Complete Hook Function Structure . . . 83

5.2 DLL Lifecycle and Deployment . . . 88

5.2.1 Injection into a Running Process . . . 88

5.2.2 Injector Launch Mode . . . 89

5.2.3 DLL Ejection . . . 91

5.3 Summary . . . 98

Appendix A Code-Cache-Based DBI Frameworks 99

A.1 Core Architectural Components . . . 99

A.2 The Execution Lifecycle: The JIT Loop . . . 100

A.3 Comparison with Inline Hooking . . . 101

A.4 The Client Interface: Instrumentation vs. Analysis Functions . . . 101

A.5 High-Level Abstraction API: Intel Pin . . . 103

A.6 Low-Level API: DynamoRIO . . . 104

A.7 Summary of Code-Cache-Based DBI Frameworks . . . 105

Appendix B How a Program Opens a File on Windows 107

B.1 The Layers of a File Open . . . 107

B.2 Parameters and Result . . . 108

Appendix C Preliminary Experiments 109

C.1 Scenario 1: Single-Threaded Hooking and Process Injection/Ejection Mechanics 109
C.2 Scenario 2: Multi-Threaded Hooking and Concurrency Management 110
C.3 Scenario 3: Instruction-Level Assembly Tracking via Code-Cache-Based DBI Engines 111

Chapter 1

Introduction

1.1 Motivation

Many industrial processes are supervised and controlled by software. Operators and security analysts often need to know what this software does at runtime, and sometimes they also need to change how it behaves. For example, they may want to block a process from accessing specific resources, measure how long a critical routine takes, or count how often a function fails.

When such an application is built on a very large codebase that has grown over many years, has been changed by different teams, and is poorly documented, even adding a single log line or a new security check can be a slow task: it requires understanding a huge codebase and rebuilding the whole application every time.

This task can be simplified with dynamic binary instrumentation (DBI). At its core, DBI is the technique of inserting additional code into a process while it is running, in order to observe or modify its behavior without needing access to the original source code.

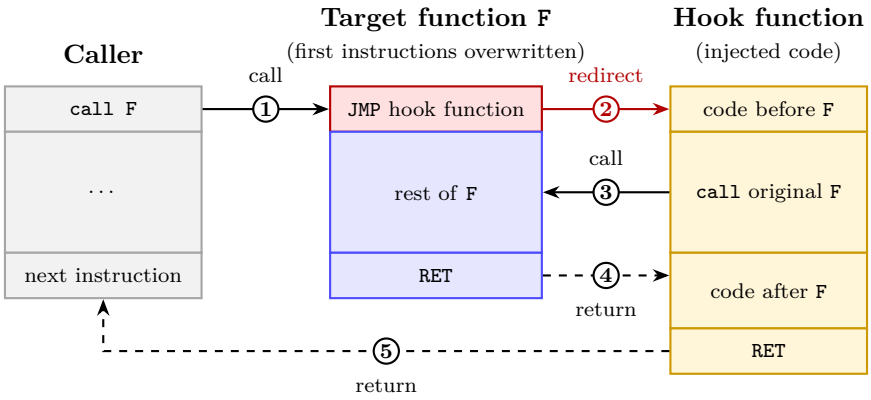
This thesis distinguishes two architectural paradigms of DBI frameworks, shown in Figure 1.1:

- *Inline Hooking:* the first instructions of a target function `F` are patched in memory with a jump instruction. The jump sends the execution to a custom injected function, called the *hook function*, which runs in place of the target function: it can run code before and after calling the target function, or skip the call, and then it returns to the caller. The jump and the hook function together are called a *hook*.

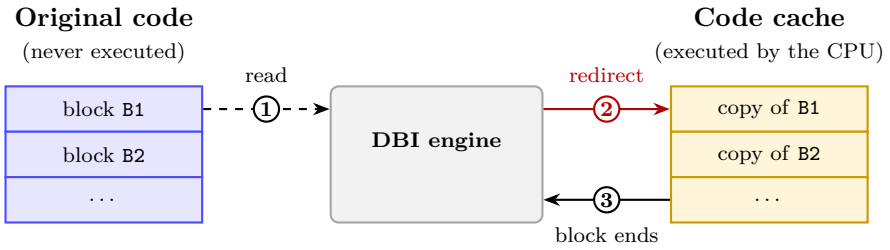
  Seen from the caller, the hook function takes the place of `F`; seen from `F`, it is the caller. The hook function therefore works like a wrapper of `F`: it receives the arguments of

`F`, runs its own code before and after, and returns the result of `F` to the caller.

- *Code-Cache-Based DBI:* instead of modifying the target code in place, the DBI engine copies the instructions, translates them, inserts the instrumentation code requested by the developer, and executes them in a separate memory area called the *code cache*.



(a) Inline hooking: the code of the target function is patched in memory. Step 3 is simplified: the overwritten instructions of `F` still run before the rest of `F` (Figure 2.1).



(b) Code-cache-based DBI: the original code is left intact and only its copies are executed.

Figure 1.1: How the two DBI paradigms redirect the execution of the target program.

Writing the code of a hook by hand is difficult and error-prone. The logic that the developer wants to add, for example a check that blocks the access to a resource, is only a small part of this code. The hook function runs inside the target process, in place of the target function and on the threads of the program, so the rest of its code must make sure that the hook never crashes the program and never changes its behavior in ways the developer did not intend. Moreover, the hooks are installed and removed while the other threads of the program keep running, and some of these threads may be inside a hook function at that moment (Section 2.3.1). None of this code is part of the task, yet much of it is repeated in every hook function, and a single mistake in it can crash the program that the developer only wanted to observe or change.

A second problem is that instrumentation tasks differ from each other: some only observe the calls to a function, others change their arguments or results, and others block them. Each task needs a different structure of the hook function. For example, to block a call, the hook function must skip the target function. If the target function never returns, such as the function that ends the process, no code can run after the call. The safety code described above must therefore be adapted to each structure, and cannot simply be copied from one hook function to another (Section 5.1.5).

A third problem is that each inline hooking framework has its own API, and the frameworks also differ in which of the safety problems above they solve themselves and which ones they leave to the developer (Chapter 3). Moving a hook from one framework to another, for example to use a feature that only the other framework offers, is therefore not a matter of renaming function calls: the developer must also write again the safety code that the new framework does not provide.

These difficulties show a gap between the *what* and the *how* of instrumentation. The *what* is short and easy to state: which functions to intercept and which behavior to apply to each of them, for example to observe, change or block their calls. The *how* is the much longer, low-level code that turns this description into working hooks: code that makes the hooks safe, follows the structure that each behavior needs, and uses the API of one specific framework. With an inline hooking framework alone, the developer writes this code by hand, adapts it for every new task and rewrites part of it for every other framework.

1.2 Research Question and Running Example

The gap between the *what* and the *how* leads to the question that this thesis answers:

> *Can a concrete DBI task be written as a declarative policy, and can the framework-specific instrumentation code be generated from it automatically, for more than one framework, so that the behavior of the target process changes according to what the policy states? And at what runtime cost?*

The work of this thesis answers the research question through the design and implementation of the *HookDesigner* tool. With this tool, the developer describes the *what* in a *policy* written in a domain-specific language (DSL) and HookDesigner generates the framework-specific *how* (the code) from it.

To keep the following chapters concrete, the thesis follows a single example from start to end, the *running example* selected as the *concrete DBI task* mentioned in the research question.

A Windows program opens a set of files. The goal is to stop this program from opening some of these files, while all the other files still open normally, and to write a log line for every blocked attempt. The source code is not touched and the target program is not rebuilt: the new behavior is added by using DBI.

1.3 Content of the Thesis

Chapter 2 introduces the foundational concepts of DBI within a Windows environment necessary to understand the proposed work.

Chapter 3 analyzes three existing inline hooking engines for Windows: Microsoft Detours, MinHook, and Frida-Gum. These engines are the candidate compilation targets of HookDesigner, which uses Detours and Frida-Gum (Section 4.1.3).

Chapter 4 presents the design, architecture, and implementation of HookDesigner, focusing on how it generates and builds a DBI payload starting from a DSL policy.

Chapter 5 follows this payload into the target process, where an error in its code would crash the target: it shows how the generated code prevents possible errors, and how the payload is injected and later removed while other threads may still be inside the hook functions.

Appendix A describes the code-cache-based engines Intel Pin and DynamoRIO, Appendix B explains how a Windows program opens a file, and Appendix C reports the first experiments that I ran with the inline hooking and the code-cache-based engines before designing HookDesigner.

Chapter 2

Background

This chapter begins by introducing the foundational concepts of DBI within a Windows environment necessary to understand the proposed work. It first explains how instrumentation code enters a target process through dynamic-link libraries (DLLs) and process injection, and how this code finds the address of a function inside the process (Section 2.1), and then the application binary interface (ABI) rules that this code must follow when it runs in place of a target function (Section 2.2). Then it explains how inline hooking works and which problems it must solve in a running process (Section 2.3), and it briefly presents code-cache-based DBI (Section 2.4). Finally, it introduces the logging and metrics libraries that the hooks generated by HookDesigner use to report what they observe at runtime (Section 2.5).

2.1 Dynamic-Link Libraries

The hook function in Figure 1.1a must be in the memory of the target process, or the jump written into `F` could not reach it. In Windows, this code is usually brought into a running process as a *DLL*, through *process injection*. A DLL is a binary module that contains functions, resources, and data that processes can load and use at runtime [Lev00, Ch. 10]. A DLL is built like a program: the developer writes its functions in a language such as C or C++, and the compiler and the linker turn them into machine code. The hook function is one of these functions, so it is ordinary compiled code. This thesis calls the DLL that is injected to instrument a process the *payload*.

2.1.1 Address Space Isolation

In modern operating systems, every running application is allocated an isolated, private virtual address space. This architectural boundary prevents one application from directly reading, writing, or corrupting the code or memory space of another.

A DLL does not break this isolation, because it does not run as a separate process. When a process loads a DLL, the operating system loader *maps* the DLL into the address space of that process: it places the code and data of the DLL file in a free range of addresses of that space [Yos+17]. From then on, the DLL is part of the process, like the code of the program itself: its functions are called by the threads of the process and have the same access to the memory of the process as the program's own code.

2.1.2 DLL Injection

Target processes are unaware of the instrumentation, so the developer must force the target to load the instrumentation DLL from the outside. This is achieved through *DLL injection* [The26a]. The standard injection technique leverages the Windows API functions `OpenProcess`, `VirtualAllocEx`, `WriteProcessMemory`, and `CreateRemoteThread` targeting `LoadLibrary`, as shown in Code 2.1. The injector first calls `OpenProcess` with the process identifier (PID) of the target and receives a *process handle*, a value that the other functions take as input to act on the target process [Mic26n]. With this handle, the injector process allocates memory in the remote target, writes the absolute path of the instrumentation DLL, and spawns a remote thread pointing to `LoadLibrary`, which executes within the target process and triggers the OS loader to map the DLL. To start this thread, the injector must know the address of `LoadLibrary` in the target process: Section 2.1.3 explains why this address is the same as in the injector.

When the loader maps a DLL, it calls a function of the DLL called its *entry point*, `DllMain`, so that code in the DLL can start to run, for example to install the hooks. While `DllMain` runs, the loader holds a lock of the process, so Microsoft advises doing only simple work inside it [Mic26f]; Section 4.3.4 explains why this matters for HookDesigner.

When the instrumentation is no longer needed, the DLL first removes its hooks, writing back the original first bytes of each target function, and then the injector makes the target call `FreeLibrary`, the counterpart of `LoadLibrary`, which unmaps the DLL. The target keeps running without a restart, and its calls no longer go through the hook functions, so they no longer pay their cost.

Code 2.1: Remote-thread DLL injection.

```
// pid: PID of the target process
const char path[] = "C:\\hooks\\hook.dll";   // absolute path of the DLL to inject

// 1. Open the target with the access rights that CreateRemoteThread requires
HANDLE hProcess = OpenProcess(PROCESS_CREATE_THREAD | PROCESS_QUERY_INFORMATION |
    PROCESS_VM_OPERATION | PROCESS_VM_WRITE | PROCESS_VM_READ, FALSE, pid);

// 2. Allocate a buffer in the target and copy the path into it
LPVOID pRemotePath = VirtualAllocEx(hProcess, NULL, sizeof(path),
    MEM_COMMIT, PAGE_READWRITE);
WriteProcessMemory(hProcess, pRemotePath, path, sizeof(path), NULL);

// 3. Address of LoadLibraryA in the injector, also valid in the target
LPVOID pLoadLibrary = GetProcAddress(GetModuleHandleA("Kernel32.dll"), "LoadLibraryA");

// 4. Start a thread in the target that runs LoadLibraryA(pRemotePath)
HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0,
    (LPTHREAD_START_ROUTINE)pLoadLibrary, pRemotePath, 0, NULL);
```

2.1.3 Modules and Function Addresses

Once the DLL is in the target process, its code can install the hooks. To hook a target function `F`, this code must know the address of `F` in the address space of the target process, because it writes the jump of Figure 1.1a at that address. This subsection explains how Windows describes where a function is, and why this address is known only while the process runs.

In Windows, every DLL that a process loads are called *modules* [Mic26i]. DLLs use the portable executable (PE) format, the format of Windows executable files [Mic26o; Mic26b]. The first address of the range where the loader maps a module is the *base address* of the module. The Windows API identifies a loaded module by a *module handle*, of type `HMODULE`, whose value is exactly this base address [Mic26r]. For example, `GetModuleHandleA` receives the name of a module that the calling process has already loaded and returns its handle; if the name is `NULL`, it returns the handle of the program file [Mic26i].

Inside a module, each function is at a fixed distance from the base address. This distance is the relative virtual address (RVA) of the function: its address in memory minus the base address of the module [Mic26o]. The linker writes in the module file where each part of the module lies relative to the base address, and the loader keeps these distances when it maps the module [Mic26o]. So the address of a function is the base address of its module plus the RVA of the function. For example, a function with RVA `0x1A30` in a module mapped at base address `0x7FF6A0000000` is at address `0x7FF6A0001A30`.

The base address, instead, is not fixed. The module file states a preferred base address, but the loader might not use it [Mic26o]. Moreover, by default the Microsoft linker marks a module so that Windows can map it at a random base address when it is loaded, a feature called address space layout randomization (ASLR) [Mic26b]. If the module of the example above is mapped at another base address, the function is at another address, but its RVA is still `0x1A30`. So the address of a function is known only after its module has been loaded, while its RVA depends only on the module file.

A module can also offer some of its functions to other modules: these functions are called *exported*. The module lists them in its *export table*: the part of this table called the export address table (EAT) holds the RVA of each exported function, and other parts link a public name to it [Mic26o]. For example, the Windows DLL `ntdll.dll` exports `NtOpenFile`, a function that opens a file and that the running example hooks (Appendix B). `GetProcAddress` receives a module handle and the public name of an exported function, and returns the address of the function in the calling process [Mic26j]. This public name can differ from the name that the function has in the source code [Mic26j]. A function that is not exported, such as an internal function of a program, has no entry in the export table, so `GetProcAddress` cannot find it: its address must be computed from the base address of its module and its RVA.

ASLR has one more property, which explains how the injector of Section 2.1.2 knows the address of `LoadLibrary` in the target process. ASLR does not choose a new base address for a DLL in every process [Yos+17]. It chooses a random base address the first time the DLL is loaded [Tho20], and when other processes load the same DLL file, Windows normally maps it at the same base address [Yos+17; Che16], so that all the processes can share a single copy of the DLL in memory [Yos+17]. So `Kernel32.dll` is normally at the same base address in the injector and in the target. Therefore, the address of `LoadLibrary` that `GetProcAddress` returns in the injector (step 3 of Code 2.1) is also its address in the target, and the injector can pass it to `CreateRemoteThread`.

2.2 The Application Binary Interface (ABI)

In inline hooking (Figure 1.1a), the caller prepares the arguments for the target function `F` and executes `call F`, but the jump sends it to the hook function instead. The caller does not know about this change, so the hook function must behave towards the caller exactly as `F` would: it must find the arguments where `F` would find them, put its result where `F` would put it, and leave the registers and the stack of the thread, together called its *machine state*, as `F` would leave them. These rules are fixed by the ABI.

The ABI defines how arguments are passed to a function, where the return value is placed, who *cleans the stack*, that is, who removes the arguments from the stack after the call (the

caller or the called function, the *callee*), and which registers a function must preserve. A set of these rules is called a *calling convention*. When it compiles a function, the compiler follows the specified calling convention: for example, if the function uses a register that it must preserve, the compiler saves that register in the first instructions of the function, the *prologue*, and restores it in the last instructions before the return, the *epilogue*. The compiler applies these rules also to the hook function, which is an ordinary compiled function of the injected DLL (Section 2.1). So the hook function leaves the machine state that the caller expects only if it is declared like `F`: with the same calling convention, the same parameters in the same order and with the same types, and the same return type. The calling convention fixes the rules; the parameters and the return type tell the compiler where, under these rules, each argument and the result are placed. The next two subsections show what goes wrong when one of them differs.

2.2.1 Windows x86 (32-bit) Architecture

In 32-bit environments, multiple calling conventions coexist [Mic26d], so the hook function defined by the developer must use the same convention as the target function. The most common conventions include:

- `__cdecl`: Arguments are pushed onto the stack from right to left, and the *caller* cleans the stack. This is the default convention for C/C++ binaries.
- `__stdcall`: The standard convention for the Windows API. Arguments are pushed right to left, but the *callee* cleans the stack.
- `__fastcall`: The first two arguments are passed in the `ECX` and `EDX` registers, while the rest are placed on the stack. The *callee* must clean the stack.
- `__vectorcall`: An extended convention designed to pass several floating-point arguments in registers. The first two integer-type arguments in the parameter list, from left to right, are placed in `ECX` and `EDX`, and up to six vector arguments in `XMM0`–`XMM5`. The *callee* must clean the stack.

Across these 32-bit conventions, registers are typically divided into *caller-saved registers* (`EAX`, `ECX`, and `EDX`), which the caller assumes can be freely overwritten by the called function, and *callee-saved registers* (`EBX`, `ESI`, `EDI`, and `EBP`), which the called function is contractually required to preserve and restore before returning.

A hook function that uses a different convention from its target breaks what the caller expects. For example, the functions that the running example intercepts (Appendix B) are declared with the macro `NTAPI`, which on x86 stands for `__stdcall`, so their callers expect

the callee to clean the stack. A hook function declared `__cdecl` would return without removing the arguments, and the caller would continue with a wrong stack pointer (`ESP`).

2.2.2 Windows x64 (64-bit) Architecture

Windows x64 defines a single native calling convention [Mic26s]. The compiler accepts `__cdecl`, `__stdcall` and `__fastcall` for compatibility but ignores them; `__vectorcall` remains in effect on x64 and must be matched exactly when a target uses it. The default convention works as follows:

- Arguments are assigned to positions. Position $i \in \{1, \ldots, 4\}$ is passed in `RCX`, `RDX`, `R8`, `R9` if the argument is an integer or a pointer, and in `XMM0`–`XMM3` if it is floating point. In `f(int a, double b, int c, double d)`, `a` travels in `RCX`, `b` in `XMM1`, `c` in `R8` and `d` in `XMM3`.
- Further arguments are pushed on the stack.
- Integer and pointer results are returned in `RAX`, floating-point results in `XMM0`.
- *Volatile (caller-saved)*: `RAX`, `RCX`, `RDX`, `R8`–`R11`, `XMM0`–`XMM5`. *Non-volatile (callee-saved)*: `RBX`, `RBP`, `RDI`, `RSI`, `R12`–`R15`, `XMM6`–`XMM15`; a function that uses them must save them in its prologue and restore them before returning.

`__vectorcall` keeps the positional scheme but extends it: vector-type arguments among the first six are passed by value in `XMM0`–`XMM5`, whereas the default convention passes an `__m128` by reference and uses `XMM0`–`XMM3` only.

A hook function whose parameters or return type differ from those of its target also breaks what the caller expects, even with the same convention. If the hook function of `f` above declared `b` as an `int`, it would read `b` from `RDX`, while the caller put it in `XMM1`. In the same way, if `f` returned a `double` and its hook function returned an `int`, the hook function would put its result in `RAX`, while the caller reads it from `XMM0`.

2.3 Inline Hooking

An inline hook, or simply *hook*, involves three components:

1. *The Target:* the original function whose execution the hook intercepts. The engine overwrites the first bytes of the target function with an unconditional jump (`JMP`)

that redirects execution to the hook function. Detours, for example, writes a 5-byte jump on both x86 and x64 [Mic26e]; since the engine moves only whole instructions (Section 2.3.1), it may overwrite more than 5 bytes. These bytes are written back when the hooks are removed (Section 2.1.2).

2. *The Hook Function:* the custom function, in the injected DLL, that the jump leads to. This function receives the execution flow, inspects the target function's arguments or return value, performs the desired analysis or policy enforcement, and then dictates whether the execution of the target function should continue or be blocked.

3. *The Trampoline:* the jump overwrites the first instructions of the target function, so these instructions can no longer run in their place. The trampoline is a small block of executable memory, built by the engine. It holds the instructions that the jump overwrote, followed by a jump to the rest of the target function.

Figure 2.1 shows how the three components work together. The caller calls `F` (step 1), and the jump at the start of `F` sends the execution to the hook function (step 2). The hook function runs its *pre-logic*, the code that runs before `F`, for example to read the arguments, and then calls the trampoline (step 3). The trampoline runs the moved instructions and jumps to the rest of `F` (step 4). Since the hook function reached `F` through a `call` to the trampoline, the return address on the stack points into the hook function, so when `F` returns, it returns to the hook function (step 5). The hook function then runs its *post-logic*, the code that runs after `F`, for example to read the return value, and returns to the caller (step 6). To block a call, the hook function skips steps 3 to 5 and returns at once, so `F` never runs.

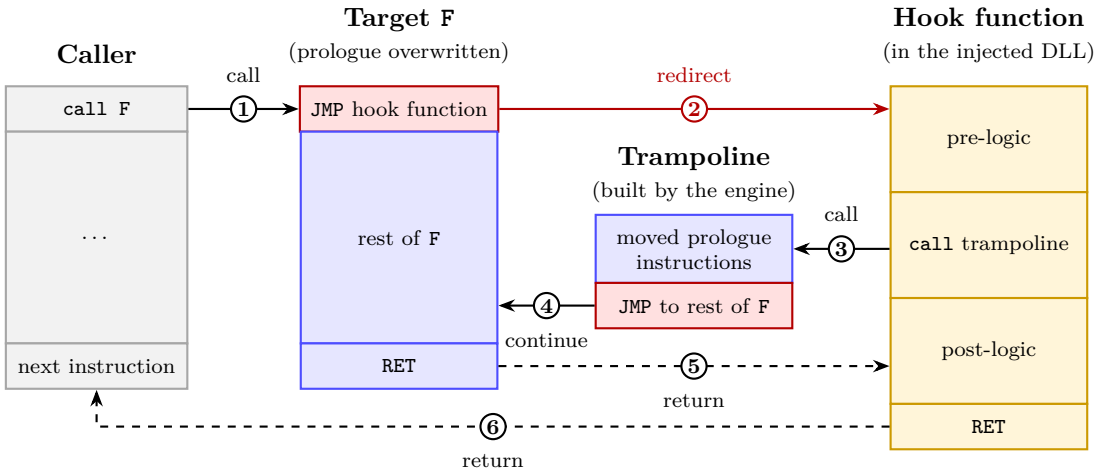


Figure 2.1: The three components of an inline hook: the target `F`, the hook function and the trampoline.

2.3.1 Challenges in Inline Hooking

An inline hooking engine changes the code of a process while the process is running, and the hook function then runs inside that process, on the threads of the program. This creates different challenges; Chapter 3 shows which of them the existing engines solve, and which ones they leave to the developer of the hook function.

1. *Instruction boundaries:* instructions do not all have the same length. On x86, the jump written into the prologue takes 5 bytes (1 byte for the opcode and 4 bytes for the relative address). If the engine blindly overwrites exactly 5 bytes, it may cut an instruction in half. For example, if the prologue starts with two 3-byte instructions, the jump overwrites the first one and only part of the second, and leaves one byte of the second instruction behind it. The trampoline would then hold an incomplete instruction and jump back to that leftover byte, so the CPU would run wrong instructions and the process would crash or behave in an unexpected way. The engine must therefore know where each instruction of the prologue ends, and move only whole instructions into the trampoline.

2. *Patching a running process:* the engine writes the jump while the other threads of the process keep running, and one of them may reach the target function at that moment. This can go wrong in two ways:

   - a thread runs the prologue while the jump is only partly written, so the CPU decodes a mix of old and new bytes as an invalid instruction and the process crashes;
   - a thread was paused by the operating system with its instruction pointer (IP) inside the bytes that are being replaced. When it continues, it starts in the middle of the new jump and runs the destination address of the jump as machine code.

3. *Unloading the DLL:* when the hooks are removed, writing the original prologue back stops *new* calls from entering the hook function. However, other threads may still be inside the hook code that they entered before. A thread is inside the hook code not only while it runs the hook function, but also while it runs the original function that the hook function called through the trampoline: when that function returns, the thread continues in the hook function (step 5 in Figure 2.1). If the DLL is unloaded at that moment, these threads run code at addresses that are no longer mapped, and the process crashes with an Access Violation. The DLL can therefore be unloaded only after every thread has left the hook code. How long this takes has no upper limit when the original target function *blocks*, that is, when it waits for an event before it returns. For example, a thread that calls `recv` waits until data arrives on a network connection, and a thread that calls `WaitForSingleObject` with no timeout

waits until another thread signals the object. If the target function blocks, or calls a function that blocks, a thread may stay inside the hook code for minutes or hours, and during all this time the DLL cannot be unloaded.

4. *Re-entrant calls:* the hook function itself may call the target function, directly or through a library function that calls it internally. Such a call passes through the jump again and enters the hook function a second time, on the same thread, before the first execution of the hook function has finished: the hook function is *re-entered*.

   For example, suppose that the hook function of the running example (Section 1.2) reads the list of the files to block from a configuration file the first time it runs. Opening the configuration file is itself a file open, so it reaches the target function and enters the hook function again; the list is still not loaded, so this second execution opens the configuration file again, and so on. Each nested call takes a new frame on the stack of the thread, so the stack soon runs out, and the process crashes with a stack overflow.

   Re-entrancy can also be *indirect*: if the hook function of a function A calls another target function B, and the hook function of B calls A, the two hook functions call each other with no end.

2.4 Code-Cache-Based DBI

Inline hooking works at function level: it patches the entry of selected functions and leaves the rest of the program untouched. Code-cache-based DBI engines, such as Intel Pin [Luk+05] and DynamoRIO [BGA03], work instead at instruction level: the original code of the application is never executed directly by the CPU. A *dispatcher* passes each new block of code to a just-in-time (JIT) compiler, which translates it, inserts the instrumentation code requested by the developer, and stores the result in a *code cache* (Figure 1.1b). The CPU then runs the translated block from the cache, and blocks that are already translated are reused.

This approach allows a very detailed analysis, such as logging every conditional branch taken by a program. However, the engine must translate every block of code that the program runs, even when the block is not instrumented. Each block is translated only once and then reused, but the first translation, the dispatcher, which runs again when a translated block cannot jump directly to the next one, and the saving and restoring of the state of the program all take time (Appendix A), so the runtime overhead is much higher than with inline hooking.

HookDesigner supports inline hooking engines (Section 4.1.3), and the integration for code-cache-based DBI is left to a future extension of this work.

Appendix A describes the architecture of this paradigm in detail, compares it with inline hooking, and presents the APIs of Intel Pin and DynamoRIO.

2.5 Logging and Metrics

Often the goal of inline hooking is intercepting a function in order to monitor it, and what it observes must then reach the analyst. The data that a running program reports about its own execution is called *telemetry*. The two most common forms are *logs*, such as a specific message written each time a file is opened, and *metrics*, such as how often a call fails or how long a routine takes. HookDesigner generates the code for both (Section 4.4) on top of two existing libraries: *Spdlog* [Mel26] for logs and *Prometheus* [The26b; Mp26] for metrics. This section explains how these libraries work.

2.5.1 Logging with Spdlog

A *log* is a text message with a related *severity level*. Spdlog uses six levels, from the least to the most severe: trace, debug, info, warning, error and critical.

Spdlog builds the text of each message with the formatting library `fmt` [Zf26].

A *logger* is the object that receives the messages: it discards those below a minimum level and passes the others to one or more *sinks*, the objects that write messages to a destination. Spdlog provides a sink that writes to a file (`basic_file_sink_mt`) and one that sends each message as a network packet. This is the most important difference with most of the logging engines available on Windows that include only file sink. Industrial deployments frequently aggregate telemetry across distributed nodes into monitoring systems (such as Promtail, Grafana Loki), and with Spdlog's built-in sinks we can do this without writing any additional code.

A *synchronous* logger runs the sinks in the thread that logs the message. Writing to a disk or to the network is usually a slow operation, and the thread waits until the write is done. In a hook function, this wait would be added to every intercepted call that logs a message.

An *asynchronous* logger separates the two tasks with a producer–consumer scheme. The thread that logs (the producer) only formats the message and copies it into a queue in memory; one or more *background threads* (the consumers) are created to take the messages from the queue and pass them to the sinks. The disk and network writes therefore happen on the background threads without slowing down the threads of the target.

In Spdlog, `spdlog::init_thread_pool(q_size, thread_count)` creates the queue and the background threads, and a `spdlog::async_logger` sends its messages to them. The

queue is a circular buffer with a fixed number of slots, `q_size`.

A mutex protects the queue, so threads that log at the same time take turns, but each thread holds it only for the time needed to copy one message.

When messages arrive faster than the consumer threads can write them, the queue fills up. The *overflow policy* of the logger then decides what happens to a new message:

- `block`, the default, makes the producer wait until a slot is free;
- `overrun_oldest` overwrites the oldest message in the queue;
- `discard_new` drops the new message.

Only with the last two policies does a producer never wait for the sinks. Section 4.4.3 describes the queue size, sinks and policy that HookDesigner uses.

2.5.2 Metrics with Prometheus

A *metric* is a number that summarizes many events, for example the number of calls to a function or the current length of a queue. A log keeps one message per event: to know how many calls failed in the last minute, a tool must store and parse every message. A metric gives this answer directly, and its size does not grow with the number of events.

Prometheus is an open-source monitoring system [The26b]. It follows a *pull model*: the monitored program does not send its metrics, but publishes their current values on an endpoint that answers HTTP requests, and the Prometheus server reads this endpoint at regular intervals. Each read is called a *scrape*. The server stores the scraped values as *time series*, that is, sequences of values ordered in time. HookDesigner uses three of the metric types of Prometheus:

- a *counter* is a value that can only increase, such as the number of calls or of errors;
- a *gauge* is a value that can go up and down, such as the number of items in a queue;
- a *histogram* counts observations by size. A list of increasing upper bounds divides the possible values into *buckets*, and the histogram counts how many observations fall into each bucket, together with the number and the sum of all observations. It shows how a quantity, such as the duration of a call, is distributed, and not only its average.

A metric can carry *dynamic labels*: key-value pairs that describe the runtime context of an observation. Each combination of label values is a separate time series. For

example, in the running example (Section 1.2), a counter of file opens with the label `verdict` produces the series `file_opens_total{verdict="allowed"}` and `file_opens_total{verdict="denied"}`. The set of series that share a metric name is called a *metric family*.

Dynamic labels have a cost. Every new combination of label values creates a new time series, which both the program and the Prometheus server must keep. If a dynamic label takes its values from a large or unbounded set, such as user IDs or memory addresses, the number of series grows without limit. This problem is called *cardinality explosion*, and the Prometheus naming guidelines[1] warn against such labels.

HookDesigner uses prometheus-cpp [Mp26], a C++ client library for Prometheus. A `Registry` holds the metric families of the program, and `BuildCounter()`, `BuildGauge()` and `BuildHistogram()` create a family and register it in a registry. `Family::Add(labels)` returns the metric of one combination of label values, and creates it the first time that combination appears. An `Exposer` runs a small HTTP server, inside the program, and at each scrape it serves the families of the registries attached to it (on the path `/metrics` by default).

Calling `Family::Add` also has a cost at runtime. The labels are a `std::map` of strings, so building them allocates memory on the heap. `Add` then creates a new metric object, also on the heap, takes a mutex that protects the family, and looks the labels up in a hash table. If the labels are already there, it discards the new object and returns the existing metric. When several threads call `Add` on the same family at the same time, they must wait for each other on the mutex. This waiting is called *lock contention*. Section 4.4.4 shows how HookDesigner uses these objects.

2.6 Summary

This chapter introduced the concepts used in the rest of the thesis. The ABI fixes how a function receives its arguments and which registers it must preserve, and therefore how a hook function must be declared. DLL injection places the instrumentation code inside the address space of the target process, where any error in that code affects the target itself. Inline hooking redirects single functions through a jump, a hook function and a trampoline, and in a running process it must deal with several challenges. Code-cache-based DBI rewrites every instruction instead, at a much higher runtime cost. A hook function reports what it observes through logs and metrics: an asynchronous logger such as Spdlog moves the disk and network writes off the thread of the target, and Prometheus collects counters, gauges and histograms, split by labels, from an HTTP endpoint inside

[1] https://prometheus.io/docs/practices/naming/

the program.

Chapter 3

State of the Art

Chapter 2 described inline hooking paradigm in general terms and the problems that every inline hooking engine must solve in a running process. This chapter presents three existing inline hooking engines for Windows, Microsoft Detours, MinHook and Frida-Gum, and shows how each of them handles those problems. These engines are the candidate compilation targets of HookDesigner.

3.1 Microsoft Detours and MinHook

Microsoft Detours [HB99] is the well-known standard for inline hooking on Windows x86/x64. Detours calls the hook function the *detour function*; this thesis keeps the name *hook function* for all the engines. The jump that Detours writes into the prologue of the target leads directly to the hook function, without executing intermediate assembly code as made in Frida (Section 3.2); this thesis calls this model *direct-jump hooking*.

MinHook [Kag26] is a smaller open-source library that follows the same direct-jump model and puts the same requirements on the hook function.

Since no engine code runs before it, the hook function finds the machine state exactly as the caller prepared it for the target, so it must follow the same ABI rules as the target. The hook function is ordinary compiled code, so the compiler applies these rules: its prologue and epilogue preserve the non-volatile registers. This holds only if the developer gives the hook function exactly the convention, parameters and return type of the target, as explained in Section 2.2.

The listings of this chapter hook the running example (Section 1.2). The function they intercept is `NtOpenFile`, the function of `ntdll.dll` that opens a file; Appendix B explains

this function and its parameters.

The hook function calls the original function through a pointer, which the developer declares as in Code 3.1. The type of this pointer repeats the convention (`NTAPI`), the parameters and the return type (`NTSTATUS`) of `NtOpenFile`, so that a call through the pointer follows the same ABI rules as a call to the target. In Detours, the pointer `TrueNtOpenFile` starts with the address of `NtOpenFile`, which tells Detours which function to hook; when the hook is applied, Detours replaces this address with the address of the trampoline.

Code 3.1: Detours and MinHook target function pointer declarations.

```
// Strict function pointer definition required by Detours
static NTSTATUS (NTAPI * TrueNtOpenFile)(PHANDLE, ACCESS_MASK, POBJECT_ATTRIBUTES,
  PIO_STATUS_BLOCK, ULONG, ULONG) = NtOpenFile;

// Strict function pointer definition required by MinHook
typedef NTSTATUS (NTAPI * NTOPENFILE)(PHANDLE, ACCESS_MASK, POBJECT_ATTRIBUTES,
  PIO_STATUS_BLOCK, ULONG, ULONG);
NTOPENFILE fpNtOpenFile = NULL;
```

Code 3.2 shows the structure of the hook function for the running example, `HookedNtOpenFile`. In the pre-logic phase, the hook function reads the name of the file from `ObjectAttributes` and decides whether the program may open it. For a protected file, it returns the error `STATUS_ACCESS_DENIED` at once, so the original function never runs and the file is not opened. Otherwise, it calls the trampoline: the original function opens the file, and control returns to the hook function (post-logic), which sees the result before returning it to the program.

Code 3.2: Structure of the hook function for Detours and MinHook.

```
// Detours (same for MinHook)
NTSTATUS NTAPI HookedNtOpenFile(PHANDLE FileHandle, ACCESS_MASK DesiredAccess,
  POBJECT_ATTRIBUTES ObjectAttributes, PIO_STATUS_BLOCK IoStatusBlock,
  ULONG ShareAccess, ULONG OpenOptions)
{
  // PRE-LOGIC: read the file name from ObjectAttributes
  if (/* the file is protected */)
    return STATUS_ACCESS_DENIED;   // the original function never runs

  // Call to the trampoline
  NTSTATUS status = TrueNtOpenFile(FileHandle, DesiredAccess, ObjectAttributes,
    IoStatusBlock, ShareAccess, OpenOptions);

  // POST-LOGIC: for example, write a log line with the file name and the status

  return status;
}
```

3.2 Frida-Gum

Frida-Gum [Fri26] is the underlying low-level C library of the Frida instrumentation toolkit, representing a modern, multi-architecture approach to inline hooking.

At the core of Frida-Gum is a centralized execution *dispatcher*, a C function of the engine that automatically handles the entire hook execution flow.

When a target function is called, the jump in its prologue does not lead to the hook function, but to the *entry thunk*, a short piece of assembly code that Frida-Gum generates at runtime and that calls the dispatcher. The dispatcher is compiled C code, so it may overwrite the volatile registers (Section 2.2), which still hold the arguments that the caller prepared for the target. For this reason, the entry thunk first saves all the registers of the thread into a `GumCpuContext` structure, and only then calls the dispatcher.

The Interceptor API of Frida-Gum offers two modes: *Invocation Listener mode* and *Function Replacement mode*.

In Invocation Listener mode, the developer writes two callbacks, `on_enter` and `on_leave`, which run before and after the original function. The callbacks do not need an ABI-compatible signature (Section 2.2): they receive a `GumInvocationContext` pointer and read or change arguments and the return value by index (`gum_invocation_context_get_nth_argument`). However, on Windows x64 this index-based access reads only general-purpose registers (`RCX`, `RDX`, `R8`, `R9`), so it cannot read floating-point arguments, which are passed in `XMM0`–`XMM3` (Section 2.2.2). For this reason, HookDesigner uses only Function Replacement mode (Section 4.1.3).

3.2.1 Function Replacement Mode

To overcome the limitations of index-based parameter extraction, Frida-Gum exposes *Function Replacement mode* via `gum_interceptor_replace` and `gum_interceptor_replace_fast`.

1. **Standard Replacement (`gum_interceptor_replace`):** Execution passes through Frida-Gum's `enter_thunk`, which saves the registers into `GumCpuContext` and calls the dispatcher. However, instead of invoking high-level callback listeners and then jumping to the original function, the dispatcher redirects execution to the hook function, which Frida-Gum calls the *replacement function* (the `replacement_function` argument of `gum_interceptor_replace`). It is an ordinary C/C++ function, as with Detours or MinHook. Before jumping to it, the entry thunk restores all the registers saved in `GumCpuContext`, so the hook function finds the arguments exactly where the caller placed them. The thunk and the dispatcher therefore do not free the

developer from the ABI: as with Detours, the hook function must have exactly the convention, parameters and return type of the target (Section 2.2). In return, the compiler reads every argument from the right register, including the floating-point ones.

2. **Fast Replacement (`gum_interceptor_replace_fast`):** Emits a direct assembly `JMP` instruction from the target prologue straight to the hook function, bypassing Frida-Gum's dispatcher creation entirely, yielding an execution flow identical to traditional frameworks like Detours and MinHook.

In Function Replacement mode, the hook function of the running example has the same signature and the same structure as the one for Detours in Code 3.2: the pre-logic, the call to the original function through the trampoline pointer `TrueNtOpenFile`, and the post-logic. The code that differs is the code that registers the hook, which Section 3.5 shows for both engines.

3.3 Hook Execution Flow

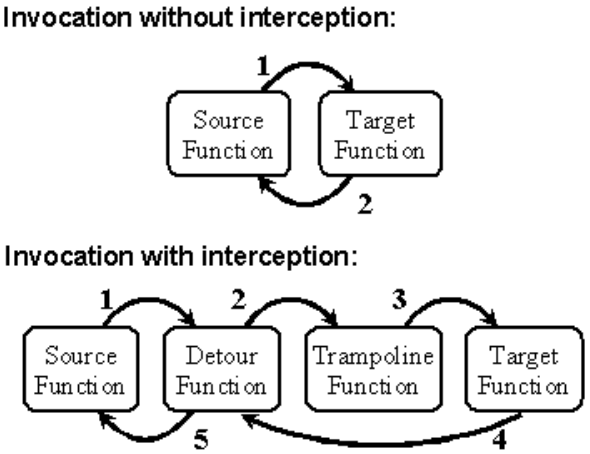
While all the presented frameworks use inline prologue patching, the role of the engine in the execution flow of a hook differs:

- **Passive Dispatch (Detours, MinHook, Frida Fast Replacement):** Overwrites the target prologue with a direct assembly jump (`JMP`) to the hook function. After the hook is installed, no engine code runs during a call: the hook function does all the work.
- **Hybrid Dispatch (Frida Standard Replacement):** Combines an active engine dispatcher with user-controlled execution routing. The engine intercepts execution via its dispatcher, but then yields execution control to the hook function, which has the same structure seen in passive dispatch frameworks.

To visually illustrate these operational differences, Figure 3.1 contrasts the direct, user-managed routing (Figure 3.1a) with the dispatcher-mediated execution path (Figure 3.1b).

3.3.1 Advantages of Engine-Mediated Dispatch

Frida-Gum employs a framework dispatcher in Standard Replacement to deliver useful capabilities that traditional frameworks do not automatically provide:



(a) Execution flow of Detours, MinHook and Frida-Gum Fast Replacement (passive). Source: Detours documentation [Mic26e].

Host Process: Invocation of `TargetFunction()`

JMP

Entry: `enter_thunk` and Dispatcher

- **CPU State Preservation:** Saves all registers into `GumCpuContext`
- **Thread Tracking:** Increments the active invocation counter in `GumFunctionContext`
- **Re-entrancy Check:** Runs the original function directly if the newest stack entry of the thread is for the same target; otherwise pushes a new entry
- **CPU State Restoration:** Restores the registers from `GumCpuContext` and jumps to the hook function

Hook Function: `HookedTargetFunction`

- **Hook Logic:** Runs its pre-logic, calls the original function through the trampoline and runs its post-logic

Exit: `leave_thunk` and Dispatcher

- **CPU State Preservation:** Saves all registers into `GumCpuContext`
- **Thread Tracking Completion:** Decrements the active invocation counter
- **Re-entrancy Tracking Completion:** Removes the entry of this call from the stack of the thread
- **CPU State Restoration:** Restores the registers from `GumCpuContext` and returns to the caller

RET

Host Process: Control Returned to Caller

(b) Execution flow of Frida-Gum Standard Replacement (hybrid).

Figure 3.1: A comparison of the execution flows of a hook.

1. **Built-in Thread Tracking for Safe Ejection (`gum_interceptor_flush`):** To unload the DLL safely (Section 2.3.1), the instrumentation code must know when every thread has left the hook function. Under passive dispatch (Detours, MinHook) and Frida's Fast Replacement, the engine does not count threads. Frida-Gum's dispatcher, instead, keeps the counter itself inside `GumFunctionContext`: it increments it in `enter_thunk` and decrements it in `leave_thunk` (Figure 3.1b), so a thread is counted from its entry until it returns to its caller, the call to the original function included. The instrumentation detachment logic can call `gum_interceptor_flush()` in a loop. When it returns `TRUE`, no thread is still inside the hooks, and the DLL can be unloaded.

2. **Built-in Protection Against Re-entrancy:** Under passive dispatch (Detours, MinHook) and Frida's Fast Replacement (`gum_interceptor_replace_fast`), the engine does nothing against re-entrant calls (Section 2.3.1): every call to the target function passes through the jump and reaches the hook function, and only the developer can stop the loop. Frida-Gum's dispatcher, instead, keeps for each thread a stack with one entry for each call that is running a hook function; the entry is removed only when the call returns to its caller (Figure 3.1b). When a target function is called, the dispatcher looks only at the newest entry: if it belongs to the same function, the dispatcher skips the hook function and runs the original function directly through the trampoline [Fri26]. This stops the *direct form* of re-entrancy, where the hook function calls its own target. It does not stop the indirect form: if the hook function of a function A calls another target function B, the newest entry belongs to A, so the hook function of B runs; if this one calls A, the newest entry belongs to B, so the hook function of A runs again, and the loop goes on. The indirect form does harm even when it does not loop: the hook function of B runs its logic and its telemetry for a call that the program never made, because the hook function of A made it.

   The check has a second limit: since the entry stays on the stack until the call returns, it stays there also while the original function runs. If the original function calls itself (recursive function), the nested call finds the entry as the newest one and is skipped too, so the hook function sees only the outermost call.

3.4 Instruction Boundaries and Disassemblers

To avoid cutting an instruction in half (Section 2.3.1), every inline hooking engine needs a disassembler that finds the length of each instruction in the prologue of the target function. A disassembler that computes only this length, without analyzing what the instruction does, is called a length disassembler engine (LDE). Before writing the jump, the engine

decodes the prologue one instruction at a time. It then copies whole instructions into the trampoline until it has moved at least as many bytes as the jump needs, and fills the remaining bytes in the original function with `NOP` instructions. The engines differ in the disassembler they use, and this choice changes the size and the dependencies of the engine:

- *Microsoft Detours* relies on its own internal decoder optimized specifically for standard compiler prologues.
- *MinHook* utilizes a highly optimized, custom-built Hacker Disassembler Engine (HDE32/HDE64) to calculate instruction lengths, rather than fully analyzing instruction semantics.
- *Frida-Gum* must handle the instruction sets of several architectures. To accomplish this, it statically links against Capstone, a comprehensive and heavy multi-architecture disassembly framework, significantly increasing the size of the resulting DLL.

3.5 Patching a Running Process

Section 2.3.1 showed that the engine writes the jump while the other threads of the process keep running, so a thread may run a half-written prologue, or continue inside the bytes that are being replaced. This section presents the techniques that prevent these dangers, and shows which of them each engine performs itself and which it leaves to the developer.

Engines patch a running process following one of two models. In the *pessimistic* model, the engine suspends the other threads of the process before it writes to the target, so that no thread runs while the prologue is being patched. Suspending the threads is not enough on its own: a suspended thread may have its IP inside the bytes that are being replaced. For this reason, the engine reads the registers of each suspended thread (`GetThreadContext`). If the IP of a thread falls inside the replaced bytes, the engine moves it to the matching copied instruction in the trampoline. When the hook is removed, the engine does the opposite: it moves a thread that is running the copied instructions in the trampoline back to the matching instruction of the restored prologue. In the *non-blocking* model, the engine does not suspend threads and does not read their IP: it makes the code pages writable and overwrites the prologue while the other threads keep running, so it prevents neither of the two dangers of Section 2.3.1. It accepts them because the chance that they happen is very small: the engine overwrites only the first few bytes of the function, a thread runs them very quickly, and they are only a small part of the code of the function, so a thread would have to be running them at the very moment of the write.

Frameworks tailored natively for Windows, such as *Microsoft Detours* and *MinHook*, follow the pessimistic model, but they differ in which threads are suspended.

Microsoft Detours groups hook attachments/detachments into a *transaction*, a set of tasks that the engine applies together, and lets the developer manage it explicitly [Mic26e]. The developer begins a transaction (`DetourTransactionBegin`), enlists every thread to be inspected via `DetourUpdateThread`, queues the hook attachments or detachments (`DetourAttach`, `DetourDetach`), and commits the transaction (`DetourTransactionCommit`). `DetourUpdateThread` suspends the enlisted thread at once, and `DetourAttach` already builds the trampoline, but the jump is written into the target only during the commit. The commit then relocates the instruction pointers of the suspended threads, flushes the CPU instruction cache, and resumes the threads. The transaction is *all-or-nothing*: if any queued operation has failed, the commit cancels the whole transaction and no hook is applied. The developer can also cancel it explicitly with `DetourTransactionAbort`, which frees the trampolines and resumes the suspended threads, in case of a fault before reaching the commit.

Code 3.3 shows a transaction that attaches the `NtOpenFile` hook of Code 3.2. Detours has no function that finds the threads of a process, so the developer must collect their handles before the transaction begins (`otherThreads` in the listing). The list leaves out the thread that runs the transaction: Detours does not suspend it, because a thread that suspends itself would never reach the commit.

Code 3.3: Attaching the `NtOpenFile` hook of Code 3.2 in a Detours transaction.

```
DetourTransactionBegin();

// Enlist every other thread of the process: each one is suspended at once
for (HANDLE hThread : otherThreads)
  DetourUpdateThread(hThread);

// Queue the hook: the trampoline is built now, the jump is not written yet
DetourAttach(&(PVOID&)TrueNtOpenFile, (PVOID)HookedNtOpenFile);

// Write the jump, move the instruction pointers, flush the cache and resume
// the threads; if a queued operation failed, cancel the transaction instead
DetourTransactionCommit();
```

MinHook adopts the same underlying mechanism but hides the transaction boundaries from its public API. Internally, when `MH_EnableHook` or `MH_DisableHook` is invoked, the library enumerates all the threads of the process, suspends them, performs instruction-pointer relocation, applies the patch, and resumes the threads.

Frida-Gum, in contrast, follows the non-blocking patching model on Windows [Fri26]. Unlike in Detours, the transaction is not automatically all-or-nothing: its transactional API (`gum_interceptor_begin_transaction` and `gum_interceptor_end_transaction`)

is only a boundary: the hooks registered with `gum_interceptor_replace` or reverted with `gum_interceptor_revert` between the two calls are applied together when the transaction ends. Each `gum_interceptor_replace` returns its own error at once, and the hooks that succeeded are still applied when the transaction ends. There is also no call to cancel a transaction; to undo staged hooks, the developer must revert them one by one. When the transaction ends, Frida-Gum makes the pages that hold the target prologues readable, writable and executable, and writes the jump into them with ordinary memory writes, one part at a time, while the other threads keep running. It does not enlist threads and does not inspect or relocate their IP, so the write is not atomic for a thread that runs the prologue at the same moment.

Code 3.4 registers the same `NtOpenFile` hook with Frida-Gum, and can be compared with the Detours transaction of Code 3.3.

Code 3.4: Registering the `NtOpenFile` hook in a Frida-Gum transaction.

```
gum_interceptor_begin_transaction(interceptor);

GumReplaceReturn r = gum_interceptor_replace(interceptor,
  targetNtOpenFile,
  (gpointer)HookedNtOpenFile,
  NULL,
  (gpointer*)&TrueNtOpenFile);

if (r != GUM_REPLACE_OK) {
  // the error is returned at once: revert the hooks staged so far, one by one
}

// Write the jumps while the other threads keep running
gum_interceptor_end_transaction(interceptor);
```

3.6 Summary

To conclude the discussion on inline hooking, Table 3.1 provides a summary comparison of the three analyzed frameworks: Microsoft Detours, MinHook, and Frida-Gum.

Appendix C reports the preliminary experiments that I ran with these engines before designing HookDesigner.

Feature	Detours	MinHook	Frida-Gum
Engine Role	Passive	Passive	Hybrid (`replace`) / Passive (`replace_fast`)
Disassembler	Built-in decoder	HDE32 / HDE64	Capstone (full semantic decoder)
Patching Concurrency	explicit thread enlistment & IP relocation	automated thread suspension & IP relocation	Non-blocking patching (plain writes while threads run, no IP relocation)
Thread-Safe Ejection	Manual tracking (counter written by the developer)	Manual tracking (counter written by the developer)	Built-in tracking (`gum_interceptor_flush`)
Re-entrancy Protection	None (left to the developer)	None (left to the developer)	Direct form only (`replace`); none (`replace_fast`)
Architectures	x86, x64, ARM, ARM64, IA64	x86, x64	x86, x64, ARM, ARM64, MIPS

Table 3.1: Comparison of the analyzed inline hooking frameworks.

Chapter 4

From Policy to Instrumentation DLL

This chapter presents the design, architecture, and implementation of HookDesigner: a DBI code generator for Windows, introduced in Chapter 1.

All the examples of this chapter come from the running example of Section 1.2, whose goal is to stop a process from opening some files. Chapter 3 presented DBI code examples written by hand; this chapter shows how HookDesigner generates this code from a DSL policy.

4.1 Design Principles and Architecture

Section 1.1 described the gap between the *what* of an instrumentation task (which functions to intercept and which behavior to apply to each of them) and the *how* (the low-level, engine-specific code that implements it). HookDesigner tries to close this gap by using a *multi-target transpilation* process.

In computer science, a *transpiler* (or source-to-source compiler) is a program that translates an input specification or program authored in one high-level language into equivalent source code in another programming language. HookDesigner uses a transpiler that automatically translates high-level policy declarations (the what of an instrumentation task) directly into the corresponding engine-specific C++ code. This code is mainly designed to run safely inside a multi-threaded target process.

Trust model: Policy authors are trusted, for this reason DSL expressions—compiled into instrumentation DLL, which runs with the privileges of the target—are not sanitized.

4.1.1 Design Principles

- **Engine-Independent Transpilation** As analyzed in State of the Art (Chapter 3), inline hooking engines expose different APIs and put different constraints on the instrumentation code. HookDesigner keeps the parts that depend on each engine separate from the rest of the transpiler. In this way, it generates the code for every selected engine from the same policy, and a new engine can be added without changing the core of the transpiler (Section 4.3).

- **Crash Safety** As analyzed in Section 2.1.1, the instrumentation payload is mapped directly into the target application's private address space. Consequently, an unhandled fault in it can crash the entire host process. For this reason, HookDesigner automatically adds safety mechanisms to every hook function it generates (Section 5.1).

- **Thread Safety** Hooking a running, multi-threaded process introduces synchronization issues both when the hooks are installed or removed and when the DLL is unloaded (as analyzed in Section 2.3.1). HookDesigner automatically generates the code that installs and removes the hooks safely, and makes sure that the DLL is unloaded only when no thread is still running inside a hook function (Section 5.2).

- **High-Performance Telemetry** Logs and metrics (Section 2.5) show what the hooks observe, but the code that produces them runs inside the hook functions, on the threads of the target, so it must slow down the target as little as possible. HookDesigner generates this code automatically from the policy, using Spdlog for logs and Prometheus for metrics (Section 4.4).

- **Separation of Policy from Code** The user describes an instrumentation task in two separate files. A declarative YAML policy states *which* functions are intercepted and *what* each hook does, for example logging the call or blocking it. The C++ code needed by this behavior, such as the code that decides whether a call is blocked, is written in a separate C++ *logic file* (Section 4.2). Since the logic file is ordinary C++, the user can write it with the usual development tools, and the same logic file can be reused with different policies and engines.

4.1.2 The Seven-Stage Pipeline

HookDesigner handles a DBI request through a pipeline of seven stages. The pipeline is split into two execution contexts: an *offline* context, which runs on the machine of the developer (stages 1–4), and an *online* context, which runs inside the target process, on the machine where the compiled DLL is deployed (stages 5–7).

- **Stage 1 – Input:** The user provides the two input files introduced in Section 4.1.1: the DSL policy and the C++ logic file (Section 4.2).
- **Stage 2 – Parse & Validate:** The transpiler reads the policy, checks that it is valid, and checks that it agrees with the logic file (Section 4.2).
- **Stage 3 – Generate:** From the validated policy, the transpiler generates the complete C++ source code of the hook DLL, one source file for each selected engine (Section 4.3).
- **Stage 4 – Compile:** Each generated source file is compiled into a Windows DLL (Section 2.1).
- **Stage 5 – Inject:** An injector program loads the DLL into the target process (Section 2.1.2) and remotely attaches its hooks. The target can be a process that is already running, or a process that the injector starts with the DLL already loaded (Sections 5.2.1 and 5.2.2).
- **Stage 6 – Execute:** The calls to the target functions now enter the hook functions, which apply the behaviors stated in the policy: they run the code of the logic file, can deny the call, and write logs and metrics (Sections 4.4 and 5.1.5).
- **Stage 7 – Eject:** When the instrumentation is no longer needed, the injector removes the hooks and unloads the DLL only when this is safe, as required by the *Thread Safety* principle (Section 5.2.3).

Thanks to this context split, the machine of the target needs only the compiled DLL and the injector binaries: the policy and the external dependencies that the transpiler uses in the offline stages stay on the machine of the developer. This matters in industrial environments, where the machine of the target is often a restricted system on which installing new programs, such as interpreters or developer tools, is not allowed.

4.1.3 Selection of Compilation Targets

The engine analysis in State of the Art (Chapter 3) and the preliminary experiments in Appendix C directly guided the choice of instrumentation engines supported by HookDesigner.

- **Microsoft Detours:** Microsoft Detours was selected because, despite its minimalistic model (analyzed in Section 3.1), it represents the gold standard for DBI in Windows x86 and x64 environments. By delegating execution routing directly to the

hook function without intermediate layers (as illustrated in Figure 3.1a), Detours achieves near-zero runtime overhead, prioritizing execution speed over framework complexity.

- **Frida-Gum:** Frida-Gum was chosen as a complementary engine to provide modern dynamic interception capabilities. Only Function Replacement mode is used, because Invocation Listener mode cannot read floating-point arguments on Windows x64 (Sections 3.2 and C.2). This allows developers to handle floating-point parameter extraction bypassing Frida default indexing, while retaining some of Frida's features not included in Detours/MinHook (presented in Section 3.3.1).

MinHook was not selected because it uses the same direct-jump model as Detours (Section 3.1) and it is also less documented. Its main practical difference, the automatic suspension of all threads (Section 3.5), is already provided for Detours by HookDesigner.

4.2 Declarative Policy Specification and Logic File Integration

This section analyzes the two input artifacts processed by HookDesigner in Stage 1 of its pipeline (Section 4.1.2): the declarative policy file, and the C++ user logic file.

4.2.1 The Core YAML Policy DSL

The policy DSL is structured as a human-readable document expressed in YAML: a human-readable data serialization language—meaning, a format used to convert complex data structures (like objects, lists, or key-value pairs) into a plain-text format that humans can easily read.

The policy contains two top-level blocks: `configure`, which establishes global setting for the instrumentation, and `hooks`, which specifies the list of target functions to intercept along with their runtime address resolution strategies and the optional code to run before and after them, the pre-logic and the post-logic (Section 2.3).

4.2.1.1 The `configure` Block

The `configure` section declares target process parameters, the relative path to the companion C++ logic file, the instrumentation paradigm and the correspondent target engines, as summarized in Table 4.1.

Field	Type	Req.	Default	Description
`process`	string	Yes	—	Target executable filename (e.g., `filedeny_target.exe`).
`logic_file`	string	Yes	—	Relative path (from the policy file) to the C++ logic source file.
`engines`	string / list	No	`"detours"`	Instrumentation engine(s) to generate code for. (e.g., `["detours", "frida"]`).
`paradigm`	string	No	*inferred*	Hooking paradigm (e.g., `inline_hook`).
`telemetry`	object	No	`{}`	Global Telemetry settings (detailed in Section 4.4)

Table 4.1: Schema definition for the top-level `configure` block.

- `engines` accepts either a single engine name or a list of target engines. When multiple engines are declared, the transpiler generates separate, engine-specific translation C++ files in a single invocation, enabling multi-engine deployments from a single policy definition.
- `paradigm` (in the current version of HookDesigner must be inline_hook) explicitly declares the instrumentation paradigm. If specified, transpiler validates that all `engines` belong to this paradigm. If omitted, it is automatically inferred from the configuration registered in `engines.yaml` of the specified `configure.engines`.

4.2.1.2 The `hooks` List

The `hooks` section defines a sequence of hook configuration objects. Each entry specifies a target function to intercept, the module (within `configure.process`), the runtime address resolution strategy to locate its address within the specified module, and the behaviors to apply to the target function (pre-logic, post-logic, call denial, logging and metrics), as detailed in Table 4.2.

- **Address Resolution Strategies :**
  - *Export Strategy (`strategy: export`):* Resolves the function entry address dynamically via `GetProcAddress` (Section 2.1.3). When hooking C++ member functions or overloaded APIs where compiler name mangling alters the exported symbol name, the optional `symbol_name` field allows specifying the decorated symbol.

Field	Type	Req.	Default	Description
`function_name`	string	Yes	—	Target function name.
`module`	string	No	`""`	Target DLL module or `configure.process`.
`is_noreturn`	bool	No	`false`	Flag for non-returning functions.
`address_resolution`	object	No	`{strategy: "export"}`	Strategy for runtime function address lookup.
`behaviors`	object	No	`{}`	Sub-policies for metrics, logging, and pre/post-logic.
Address Resolution Configuration (`address_resolution`)				
`strategy`	string	Yes	—	Resolution method: `"export"` or `"offset"`.
`symbol_name`	string	No	`function_name`	Exported decorated symbol for `GetProcAddress`.
`rva_offset`	string / int	If offset	—	Hexadecimal RVA of the function (e.g., `"0x1A30"`).
Behavioral Sub-Policies Configuration (`behaviors`)				
`security_policy`	object	No	`{}`	Names of the pre-logic and post-logic functions, and optional call denial.
`log_rules`	list	No	`[]`	Logging rules (detailed in Section 4.4).
`performance_metrics`	object	No	`{}`	Target latency and Prometheus metrics (detailed in Section 4.4)
Call Denial Configuration (`security_policy.on_deny`)				
`return_value`	string	If non-void	—	C++ expression returned to the caller of a denied call (Section 4.2.1.2.1).
`last_error`	string	No	`null`	C++ `DWORD` expression set as the caller's last-error code.

Table 4.2: Schema definition for the `hooks` list entries

  - *Offset Strategy* (`strategy: offset`): For unexported functions, HookDesigner computes their address at runtime by adding the RVA given in `rva_offset` to the base address of the specified `module`, or of the program file if `module` is empty (Section 2.1.3).

- **Non-Returning Functions (`is_noreturn`):** this flag is automatically inferred for standard non-returning APIs (such as `RaiseException`, `ExitProcess`, and `ExitThread`). Because these functions never return to the caller, the transpiler leverages this flag during semantic validation to prevent policy authors from declaring a post-logic or non-compatible telemetry options (Section 4.4.2.3) and for correctly selecting the hook function structure designed for non-returning functions (Section 5.1.5.2).

- **`Security_policy`**: is a dictionary that names the pre-logic and the post-logic of the hook, two functions implemented in the C++ logic file (Section 4.2.2.1). Optionally, an `on_deny` entry lets the pre-logic deny the original target call (Section 4.2.1.2.1).

4.2.1.2.1 Call Denial The pre-logic can inspect and change the arguments, and the post-logic can also inspect and change the return value, but the original function is always executed. Access-control policies, such as preventing a process from opening a protected file, need a different behavior: the original function must not run, and the call must fail as if the operating system had refused it.

HookDesigner supports this through the optional `on_deny` entry of `security_policy`. When `on_deny` is declared, the pre-logic becomes a decision point: it must return `bool`, where `true` allows the call and `false` denies it. The entry has two fields (Table 4.2):

- `return_value`: a C++ expression of the target's return type
- `last_error`: an optional C++ `DWORD` expression set as the caller's last-error code (Section 5.1.3)

A denied call behaves as follows:

- the original function is not called;
- the caller receives `return_value`;
- the caller's last-error code is set to `last_error` if declared, otherwise it is left as it was before the call;
- the post-logic is skipped, since there is no original result to process;

- telemetry still runs and sees `return_value` as the result of the call, so denied calls can be logged and counted through the `__hook_denied` wildcard (Section 4.4.2.2).

During validation, the transpiler rejects an `on_deny` entry that is attached to a non-returning function, that lacks a `return_value` for a non-void function or declares one for a void function.

Code 4.1 shows the hook of the running example on `NtOpenFile`. The pre-logic `CheckFileAllowed` receives the `ObjectAttributes` argument of `NtOpenFile`, which holds the name of the file (Appendix B), and returns its verdict. A denied call returns `STATUS_ACCESS_DENIED`, the `NTSTATUS` error for a refused access. The hook declares no `last_error`, because native functions report their errors only through the returned `NTSTATUS`.

Code 4.1: YAML policy snippet using call denial on `NtOpenFile`.

```
- module: ntdll.dll
  function_name: NtOpenFile
  address_resolution: { strategy: export }
  behaviors:
    security_policy:
      pre_hook: CheckFileAllowed   # bool CheckFileAllowed(POBJECT_ATTRIBUTES
    ObjectAttributes); true = allow
      on_deny:
        return_value: "STATUS_ACCESS_DENIED"
```

4.2.2 Logic File Integration

A core design principle of HookDesigner is the separation between declarative policy and imperative logic to inject, as presented in Section 4.1.1.

To let the transpiler parse the logic file, the file must be divided with tag markers:

1. **Signatures Block** (`// @signatures-start` ... `// @signatures-end`): This block defines the exact parameter types, parameter names and order, optional calling conventions, and return types of the functions targeted by the policy.
2. **Callback Implementation Regions** (`// @<CallbackName>-start`): Each callback, a pre-logic or a post-logic, is prefixed with a dedicated tag matching the name referenced in the YAML policy's `security_policy` dictionary (`pre_hook` or `post_hook`).

The logic file is included directly via `#include` in the transpiler-generated C++ file, so it can also hold helper functions and constants that the callbacks use.

4.2.2.1 Parsing with libclang

Parsing C++ function declarations with regular expressions is fragile, because of complex types, nested pointers and calling conventions. For this reason, Stage 2 of HookDesigner uses libclang, the C interface of Clang [LLV26], an open-source C and C++ compiler; the transpiler calls libclang through its Python bindings (`clang.cindex`). Like most compilers, Clang first reads the source code and turns it into an abstract syntax tree (AST): a tree in which each node is an element of the program, such as a function declaration or one of its parameters, and stores its name and its type.

The transpiler uses libclang to build the AST of the logic file, using the `-std=c++17` standard, and traverses its nodes to perform the following tasks:

1. **Extracting Signatures:** The transpiler locates the *Signatures Block* and iterates over all the function declarations. For every function declaration within this range, it extracts:
   - The *target function name* and *return type*
   - The *calling convention* (Section 2.2) by scanning the token preceding the function identifier, looking for calling convention keywords (`__stdcall`, `__cdecl`, `__fastcall`, etc.).
   - The *parameter list*, capturing parameter names and Clang type handles (`_clang_type`).
2. **Extracting Callback Parameters:** The transpiler scans for `// @<CallbackName>-start` tag patterns and filters the Clang `FUNCTION_DECL` nodes, extracting the parameter names for cross-validation against target function prototypes, and the callback return type.

During Stage 2, the transpiler enforces the following semantic constraints:

1. **Signature Bindings:** Every `function_name` specified in the YAML policy must have a corresponding function declaration inside the Signatures Block.
2. **Callback Resolution:** Every callback identifier declared under `security_policy` (`pre_hook` and `post_hook`) must have a matching `// @<CallbackName>-start` implementation in the logic file.
3. **Parameter Matching:** Each callback parameter must match the name of a parameter in the target function's signature specified in the *Signatures Block*. Type checking and reference binding are left to the C++ compiler, because a callback may take a parameter by reference.

4. **Return Value Overriding:** For a post-logic attached to a non-void function, an optional reference parameter named `__hook_result`, of the target's return type, can be declared. It holds the value returned by the original function, so the post-logic can inspect it or override the value returned to the caller.

5. **Deny Verdict:** If the hook declares `on_deny`, its pre-logic must return `bool`: this value is the verdict that allows or denies the call (Section 4.2.1.2.1).

4.2.2.2 Callback Reusability

In some cases, sharing a pre-logic or a post-logic across multiple distinct target functions could be useful for saving time and avoiding code duplication. Because HookDesigner binds callback arguments by matching parameter names against the target function's parameters (delegating type checking and reference binding to the C++ compiler), one callback can serve every target function whose parameters have the same names.

Codes 4.2 and 4.3 show this on an extension of the running example, a *read-only mode*: instead of denying a protected file, the hook lets the program open it, but removes the rights to change it from the requested access. `NtOpenFile` and `NtCreateFile` both call their access mask `DesiredAccess` and their file name `ObjectAttributes`, so a single pre-logic, `ForceReadOnly`, serves both functions.

Code 4.2: C++ user logic file illustrating callback reusability.

```
// @signatures-start
NTSTATUS NTAPI NtCreateFile(PHANDLE FileHandle, ACCESS_MASK DesiredAccess,
    POBJECT_ATTRIBUTES ObjectAttributes, PIO_STATUS_BLOCK IoStatusBlock,
    PLARGE_INTEGER AllocationSize, ULONG FileAttributes, ULONG ShareAccess,
    ULONG CreateDisposition, ULONG CreateOptions, PVOID EaBuffer, ULONG EaLength);

NTSTATUS NTAPI NtOpenFile(PHANDLE FileHandle, ACCESS_MASK DesiredAccess,
    POBJECT_ATTRIBUTES ObjectAttributes, PIO_STATUS_BLOCK IoStatusBlock,
    ULONG ShareAccess, ULONG OpenOptions);
// @signatures-end


// IsReadOnlyFile: true if the file is on the read-only list

// @ForceReadOnly-start
static void ForceReadOnly(ACCESS_MASK& DesiredAccess, POBJECT_ATTRIBUTES
    ObjectAttributes) {
    if (!ObjectAttributes || !IsReadOnlyFile(ObjectAttributes)) return;
    if (DesiredAccess & (GENERIC_ALL | MAXIMUM_ALLOWED)) DesiredAccess |= GENERIC_READ;
      // keep the read rights
    DesiredAccess &= ~kWriteRights;
}
```

Code 4.3 attaches the same callback to both hooks.

Code 4.3: YAML policy snippet attaching a shared callback across multiple hooks.

```
- module: ntdll.dll
  function_name: NtOpenFile
  behaviors: { security_policy: { pre_hook: ForceReadOnly } }
- module: ntdll.dll
  function_name: NtCreateFile
  behaviors: { security_policy: { pre_hook: ForceReadOnly } }
```

To summarize the integration between declarative policy configuration and callback definitions, Code 4.4 presents a complete YAML policy for another extension of the running example. The hook on `NtOpenFile` combines the pre-logic verdict of call denial with a post-logic.

Code 4.4: Example of core HookDesigner YAML policy (extension of the running example).

```
configure:
  paradigm: inline_hook
  engines: [detours, frida]
  process: "filedeny_target.exe"
  logic_file: "logic.cpp"

hooks:
  - module: ntdll.dll
    function_name: NtOpenFile
    address_resolution: { strategy: export }
    behaviors:
      security_policy:
        pre_hook: CheckFileAllowed
        post_hook: CheckOpenedFile
        on_deny:
          return_value: "STATUS_ACCESS_DENIED"
```

The corresponding companion `logic_file` (`logic.cpp`) is shown in Code 4.5. It holds the signatures block and the two tagged callbacks. The post-logic `CheckOpenedFile` runs only when the pre-logic allowed the call, after the original function. The pre-logic can judge only the name that the caller passed, but once the file is open, `GetFinalPathNameByHandleW` can return its *final path*, for example the target of a symbolic link [Mic26g]. If the final path names a protected file, the post-logic closes the handle and changes `__hook_result` to `STATUS_ACCESS_DENIED`.

Code 4.5: C++ logic file (`logic.cpp`) illustrating signature declarations, tag annotations, a pre-logic verdict and a post-logic that overrides the return value (simplified).

```
#include <windows.h>
#include <winternl.h>
```

```
#include "../hd_paths.h"     // CopyObjectName
#define STATUS_ACCESS_DENIED ((NTSTATUS)0xC0000022L)

// @signatures-start
NTSTATUS NTAPI NtOpenFile(PHANDLE FileHandle, ACCESS_MASK DesiredAccess,
    POBJECT_ATTRIBUTES ObjectAttributes, PIO_STATUS_BLOCK IoStatusBlock,
    ULONG ShareAccess, ULONG OpenOptions);
// @signatures-end

// IsDeniedName: true if the last component of a path is in the deny list

// Pre-logic of NtOpenFile: true = allow
// @CheckFileAllowed-start
static bool CheckFileAllowed(POBJECT_ATTRIBUTES ObjectAttributes) {
    /* checks the name of the file with IsDeniedName */
}

// Post-logic of NtOpenFile: runs only for allowed calls
// @CheckOpenedFile-start
static void CheckOpenedFile(PHANDLE FileHandle, NTSTATUS& __hook_result) {
    if (__hook_result < 0 || !FileHandle || !*FileHandle) return;
    wchar_t path[MAX_PATH];
    const DWORD length = GetFinalPathNameByHandleW(*FileHandle, path, MAX_PATH,
                                                   FILE_NAME_NORMALIZED |
    VOLUME_NAME_DOS);
    if (length == 0 || length >= MAX_PATH) return;   // name not available: keep
    __hook_result
    if (!IsDeniedName(path)) return;
    CloseHandle(*FileHandle);
    *FileHandle = nullptr;
    __hook_result = STATUS_ACCESS_DENIED;
}
```

After signature extraction and semantic validation, for each declared hook, the transpiler matches its `function_name` against the extracted signatures, extending the hook's in-memory representation with its C++ return type, parameter list, and calling convention. In Stage 3, this merged data structure is passed to the template engine that generates the C++ code (Section 4.3).

4.3 C++ Code Generation and DLL Compilation

Once Stage 2 (Section 4.1.2) has validated the declarative policy and extracted the data within the logic file, Stage 3 and Stage 4 translate the in-memory policy representation into compilable C++ source code and build the final instrumentation DLL.

Stage 3 generates the C++ code with a *template engine*. The program gives the template engine a template and a set of named values, the *render context* (Section 4.3.2.2), and the engine renders the template. HookDesigner uses Jinja2 [Pal24], a well-known template engine for Python. The templates of HookDesigner are C++ files that also contain Jinja2 code (Section 4.3.2).

To realize the foundational principle of *engine-independent transpilation* established in Section 4.1.1, this phase must overcome a specific challenge: inline hooking engines expose fundamentally different low-level interfaces as examined in State of the Art (Chapter 3). Hard-coding engine-specific boilerplate into the transpiler would couple the transpilation process to specific frameworks, preventing the addition of new instrumentation engines without modifying the transpiler core.

To resolve this challenge, HookDesigner organizes the code generation through two different layers:

1. An *external engine registry* (`engines.yaml`) that parameterizes engine-specific template selection, cast expressions, and build arguments (Section 4.3.1).
2. A *modular template hierarchy* that factors shared instrumentation logic into engine-independent partial templates while delegating specific features to engine templates (Section 4.3.2).

4.3.1 Engine Registry

The `engines.yaml` registry is a declarative configuration file that contains all engine-specific parameters and build requirements. The transpiler loads this registry at startup via `load_engines()` and uses it to drive subsequent template rendering and compilation decisions. Crucially, the registry decouples the transpiler's core code generation logic from individual hooking backends: supporting an additional hooking engine requires only supplying its template and defining its parameters in `engines.yaml`, requiring zero modifications to the transpiler logic.

4.3.1.1 Registry Schema

Engine entries are organized under a *paradigm* top-level key that groups engines by their hooking model (e.g., `inline_hook`). This classification is meant to allow a future extension of HookDesigner to other instrumentation paradigms (such as code-cache-based DBI engines like DynamoRIO or Intel Pin, Section 2.4). Table 4.3 documents the configuration schema for engines operating under the `inline_hook` paradigm, which mandates four core parameters alongside an optional list of auxiliary system libraries.

Field	Type	Req.	Role / Description
`template`	string	Yes	Filename of the engine's root template located in `templates/` (e.g., `detours_template.cpp`).
`cast_format`	format string	Yes	Python format string for function-pointer casting during dynamic symbol resolution (e.g., `"(True{name}_t)"` or `"(gpointer)"`).
`include_path`	string (path)	Yes	Header include directory for the hooking engine.
`lib_path`	string (path)	Yes	Static library (`.lib`) file path for the hooking engine.
`extra_libs`	list of strings	No	Auxiliary Windows system libraries

Table 4.3: Schema definition for engine entries registered in `engines.yaml`.

Code 4.6 lists the registry configuration supporting Microsoft Detours and Frida-Gum.

Code 4.6: Registry definitions for Detours and Frida-Gum in `engines.yaml`.

```
inline_hook:
  detours:
    template: "detours_template.cpp"
    cast_format: "(True{name}_t)"
    include_path: "${DETOURS_INC}"
    lib_path: "${DETOURS_LIB}"
    extra_libs:
      - "User32.lib"
      - "Advapi32.lib"

  frida:
    template: "frida_template.cpp"
    cast_format: "(gpointer)"
    include_path: "${FRIDA_INC}"
    lib_path: "${FRIDA_LIB}"
    extra_libs:
      - "User32.lib"
      - "Advapi32.lib"
      - "Ole32.lib"
      - "Shell32.lib"
```

4.3.2 Jinja2 Template Architecture

Jinja2 marks its own code inside a template with tags [Pal24]. The templates of HookDesigner use three kinds of tags:

- `{{ x }}` is replaced by the value of `x`, for example by the name of a target function.
- `{% for %} ... {% endfor %}` repeats the text between the two tags once for each element of a list, for example once for each hook of the policy, and `{% if %} ... {% endif %}` keeps this text only when a condition is true.
- `{% include %}` inserts another template file at that point; the inserted file is rendered with the same render context, and it can also use the variable of the `for` loop that contains the tag.

In the listings of this chapter, all the text outside these tags is the C++ code that goes into the generated file.

A naive approach to multi-target code generation would maintain monolithic, duplicated template files for each supported engine. However, with the engines that HookDesigner supports, the majority of generated code is engine-independent. Duplicating this code across engines would introduce maintenance issues.

4.3.2.1 Two-Layer Hierarchy

To maximize code reuse while preserving engine autonomy, in HookDesigner templates are organized into a two-layer hierarchy:

1. **Engine-Specific Templates** (`detours_template.cpp`, `frida_template.cpp`): These files define the skeleton of the generated file. Each template defines module-level includes, global synchronization primitives, function-pointer typedefs, the signature and the start of each hook function (Section 4.3.3), and the exported `InitializeHooks` and `DeinitializeHooks` hooks lifecycle routines (Section 4.3.4).
2. **Shared Partial Templates** (`_*.cpp`): Reusable partial templates located in `templates/`. The engine templates insert them with `{% include %}`, so the Jinja2 code inside a partial template is rendered with the same render context as the engine template.

The two lifecycle functions belong to the engine-specific templates because their bodies cannot be shared across engines: as explained in Section 3.5, guaranteeing thread safety during hook attachment and detachment requires engine-specific code.

4.3.2.2 Render Context

The render context that the transpiler passes to the template engine has the following entries:

- `hooks`: the list of hooks extracted in Stage 2, containing the hook policy data enriched with the data derived from logic file (Section 4.2).
- `logic_file_rel`: the normalized relative filesystem path from the output C++ file to the companion user logic file (Section 4.2.2), ensuring that the generated `#include` directive resolves correctly across arbitrary build directories.
- `telemetry`: the telemetry configuration dictionary (or `None`), for managing the conditional rendering of logging and metrics C++ code (Section 4.4.1).
- `has_execution_time`: `true` if at least one hook enables `execution_time` (Section 4.4.1). It decides whether the template engine needs to declare a timer-frequency variable in the generated code for the initialization phase of the DBI DLL (Section 5.2.1.2).

In the template listings that follow, a name inside a Jinja2 tag is an entry of this render context. A dot reads a field of a value: for example, `hook.function_name` is the name of the target function of that hook.

4.3.2.2.1 Target Address Resolution Before template rendering begins, the transpiler resolves the target address lookup expression for the target function of every declared hook and stores it in `hook["address_resolution_cpp"]`. The activation routine of the hooks (`InitializeHooks`, Section 4.3.4.1) uses it. To support both public exported APIs and internal (not exported) functions, HookDesigner provides two resolution strategies, presented in Section 4.2.1.2. To maintain engine independence, the resulting pointer expression is formatted according to the active engine's `cast_format` (e.g., typed function pointers for Microsoft Detours vs. `gpointer` for Frida-Gum, as defined in Table 4.3), ensuring that templates remain decoupled from engine-specific type casting conventions.

4.3.3 Hook Function Generation

For every target function declared in the policy, HookDesigner generates a hook function named `Hooked<F>`, where `F` is the name of the target function.

While the classic inline hook function structure (Section 2.3) follows a basic three-step pattern (*pre-logic* → *trampoline dispatch* → *post-logic*), deploying such a function into a live, multi-threaded process could be dangerous: an unhandled exception inside the hook function crashes the host application, re-entrant API calls cause a stack overflow, and in-flight threads crash during DLL unloading (Section 2.3.1).

For this reason in HookDesigner, the body of each hook function extends this basic pattern with automated runtime safety defenses (Section 5.1).

Each hook function comes from two templates of the two-layer hierarchy (Section 4.3.2), shown in a simplified form in Code 4.7.

As established in Section 2.2, the hook function must follow exactly the function signature and the calling convention of the target function (`F`). The engine template writes the signature of the hook function and the start of its body, and then inserts the shared partial template `_hook_body.cpp`, which writes the rest of the body. The transpiler renders the signature from the data that Stage 2 extracts from the logic file (Section 4.2.2.1): the return type, the calling convention if one is declared, and the type and order of each parameter of `F`. In this way, the hook function has exactly the signature of the target function.

Code 4.7: Templates that generate a hook function (simplified): the start of the function in the engine templates, and the shared partial template `_hook_body.cpp`.

```
// Engine template (detours_template.cpp, frida_template.cpp)
{% for hook in hooks %}
{{ hook.return_type }} {% if hook.calling_convention %}{{ hook.calling_convention }} {%
     endif %}Hooked{{ hook.function_name }}({%- for param in hook.parameters -%}{{
    param.type }} {{ param.name }}{%- if not loop.last -%}, {%- endif -%}{%- endfor
    -%}) {
    {% include "_hook_body.cpp" %}
}
{% endfor %}

// _hook_body.cpp
{% if hook.behaviors.security_policy.pre_hook %}
{{ hook.behaviors.security_policy.pre_hook }}(/* args */);
{% endif %}

// safety code before the trampoline call
{{ hook.return_type }} __hook_result = True{{ hook.function_name }}(/* args */);
// safety code after the trampoline call

{% if hook.behaviors.security_policy.post_hook %}
{{ hook.behaviors.security_policy.post_hook }}(/* args */);
{% endif %}

// telemetry code

return __hook_result;
```

The call to the trampoline through the pointer `True<F>` splits the body into two parts. This thesis calls *pre-trampoline code* all the code that the hook function runs before this call, and *post-trampoline code* all the code that it runs after the trampoline returns. The pre-trampoline code includes the call to the pre-logic, and the post-trampoline code includes the call to the post-logic and the telemetry code.

4.3.4 Hook Lifecycle Function Generation

In addition to individual hook functions (Section 4.3.3), the generated DLL must provide the routines that install and remove the hooks specified in the policy. HookDesigner generates two exported functions with standard C linkage: `InitializeHooks`, which resolves the targets and installs the hooks, and `DeinitializeHooks`, which removes them before the DLL is unloaded.

The hooks could also be installed in `DllMain`, which the loader calls when it maps the DLL (Section 2.1.2). HookDesigner does not do so, because `DllMain` runs while the loader holds a lock of the process, and the work of these two functions is not simple: when telemetry is enabled, they start the background threads of the logger and of the metrics server (Section 2.5), and they stop them again, waiting for them to end, when the hooks are removed or when the installation fails. A thread that waits for other threads while the loader holds this lock can block the process forever.

For this reason, the `DllMain` of the generated DLL installs no hooks, and the injector calls the two exported functions itself, each through a new remote thread that runs outside `DllMain`, as detailed in Section 5.2.

4.3.4.1 Transactional Attachment and Detachment

Once the target functions are located through the resolved runtime address lookup expression (Section 4.3.2.2), the template performs hook registration using the engine-specific transactional API (Section 3.5).

Under Frida-Gum, `gum_interceptor_replace` reports a failure only through its return value, so the template turns any result other than `GUM_REPLACE_OK` into a C++ exception, which starts the rollback (Code 4.9).

Depending on the active hooking engine, the template emits the corresponding transactional attachment for declared hooks, as illustrated in Code 4.8 for Microsoft Detours and Code 4.9 for Frida-Gum.

In both listings only the `catch` handlers are left as comments; their content is described in Section 5.1.4.1.

Code 4.8: Simplified Jinja2 template logic for Microsoft Detours hook lifecycle functions.

```
extern "C" __declspec(dllexport) DWORD WINAPI InitializeHooks(LPVOID lpParam) {
    try {
        // Locate Functions
        {% for hook in hooks %}
        True{{ hook.function_name }} = {{ hook.address_resolution_cpp }};
```

```
        {% endfor %}

        // Transactional Attachment
        std::vector<HANDLE> threads = EnlistAllThreads();
        DetourTransactionBegin();
        for (HANDLE hThread : threads) {
            DetourUpdateThread(hThread);
        }
        {% for hook in hooks %}
        if (True{{ hook.function_name }}) {
            DetourAttach(&(PVOID&)True{{ hook.function_name }}, (PVOID)Hooked{{ hook.
    function_name }});
        }
        {% endfor %}
        DetourTransactionCommit();

        for (HANDLE hThread : threads) {
            CloseHandle(hThread);
        }

    } catch (const std::exception& ex) {
        //diagnostic and rollback mechanism...
    }
}

extern "C" __declspec(dllexport) DWORD WINAPI DeinitializeHooks(LPVOID lpParam) {
    try {
        // Transactional Detachment
        std::vector<HANDLE> threads = EnlistAllThreads();
        DetourTransactionBegin();
        for (HANDLE hThread : threads) {
            DetourUpdateThread(hThread);
        }
        {% for hook in hooks %}
        if (True{{ hook.function_name }}) {
            DetourDetach(&(PVOID&)True{{ hook.function_name }}, (PVOID)Hooked{{ hook.
    function_name }});
        }
        {% endfor %}
        DetourTransactionCommit();

        for (HANDLE hThread : threads) {
            CloseHandle(hThread);
        }

    } catch (const std::exception& ex) {
        //diagnostic and rollback mechanism...
    }
}
```

Code 4.9: Simplified Jinja2 template logic for Frida-Gum hook lifecycle functions.

```
extern "C" __declspec(dllexport) DWORD WINAPI InitializeHooks(LPVOID lpParam) {
    try {
        gum_init_embedded();
        interceptor = gum_interceptor_obtain();

        // Locate Functions
        {% for hook in hooks %}
        target_{{ hook.function_name }} = {{ hook.address_resolution_cpp }};
        {% endfor %}

        // Transactional Replacement
        gum_interceptor_begin_transaction(interceptor);
        {% for hook in hooks %}
        if (target_{{ hook.function_name }}) {
            GumReplaceReturn r = gum_interceptor_replace(interceptor,
                                    target_{{ hook.function_name }},
                                    (gpointer)Hooked{{ hook.function_name }},
                                    nullptr,
                                    (gpointer*)&True{{ hook.function_name }});
            // Gum reports failures only through the return code: throwing starts the
    rollback
            if (r != GUM_REPLACE_OK) {
                throw std::runtime_error(fmt::format("Frida failed to replace function
    '{}'. Error code: {}", "{{ hook.function_name }}", (int)r));
            }
        }
        {% endfor %}
        gum_interceptor_end_transaction(interceptor);

    } catch (const std::exception& ex) {
        //diagnostic and rollback mechanism...
    }
}

extern "C" __declspec(dllexport) DWORD WINAPI DeinitializeHooks(LPVOID lpParam) {
    try {
        // Transactional Detachment
        gum_interceptor_begin_transaction(interceptor);
        {% for hook in hooks %}
        if (target_{{ hook.function_name }} != nullptr) {
            gum_interceptor_revert(interceptor, target_{{ hook.function_name }});
        }
        {% endfor %}
        gum_interceptor_end_transaction(interceptor);

        g_object_unref(interceptor);
        gum_deinit_embedded();
```

```
    } catch (const std::exception& ex) {
        //diagnostic only, no rollback...
    }
}
```

While Codes 4.8 and 4.9 illustrate the simplified Jinja2 template loops used to generate hooks attachment and detachment, deploying these routines into a live, multi-threaded process requires additional safeguards to coordinate thread synchronization and handle telemetry subsystems. The complete definition of `InitializeHooks` and `DeinitializeHooks` is detailed in Section 5.2.

4.4 HookDesigner Telemetry

In an industrial process, a critical function can be called many times per second by several threads of the target at the same time. When instrumenting such a process, where performance is essential, telemetry cannot come at the expense of host responsiveness.

To overcome these constraints and fulfill the *High-Performance Telemetry* design principle introduced in Section 4.1.1, HookDesigner implements the following solutions:

1. **Asynchronous Logging (Section 4.4.3):** An asynchronous logger powered by Spdlog: the thread of the target only copies each message into a circular buffer, and it never waits for the disk or the network.

2. **Real-Time Performance Metrics (Section 4.4.4):** A native Prometheus metrics subsystem based on `prometheus-cpp`, exposed through an embedded HTTP server.

Section 2.5 explains how these two libraries work; this section describes how HookDesigner configures them and generates the code that uses them.

Before generating code, HookDesigner processes telemetry policy rules during transpilation: Section 4.4.1 defines the declarative YAML schema and static validation constraints.

4.4.1 Telemetry Policy Specification

Telemetry configuration is split into two distinct components: process-wide parameters declared in the top-level `configure.telemetry` block, and hook-specific behavioral policies declared under each hook entry's `behaviors` mapping.

4.4.1.1 Global Telemetry Configuration

The top-level `configure.telemetry` mapping establishes the general settings for the logging framework and the metric server endpoint. Table 4.4 outlines the schema definitions for this block.

Field	Type	Req.	Default	Description
`logger`	string	No	`null`	Logging engine identifier (currently supports `"spdlog"`).
`logger_config`	object	No	`{}`	Log configuration, if `logger` is not `null`.
`metrics`	string	No	`null`	Metrics engine identifier (currently supports `"prometheus"`).
`metrics_config`	object	No	`{}`	HTTP exposer configuration, if `metrics` is not `null`.
Logger Settings (`logger_config`)				
`type`	string	No	`"file"`	Log destination: `"file"` (local disk) or `"udp"` (remote Syslog).
`host`	string	No	`"127.0.0.1"`	Target host IPv4 address only if `type: "udp"`.
`port`	integer	No	`1514`	Target UDP listening port only if `type: "udp"`.
`file`	string	No	`"hook_log.txt"`	Destination file path only if `type: "file"`.
`level`	string	No	`"info"`	Minimum severity filter (`trace`, `debug`, `info`, `warn`, `error`, `critical`).
Metrics Exposer Endpoint (`metrics_config`)				
`endpoint`	string	No	`"127.0.0.1:8080"`	Address and port bound by the embedded Prometheus HTTP server.

Table 4.4: Schema definition for the top-level `configure.telemetry` block.

4.4.1.2 Hook-Level Telemetry Behaviors

Each hook can declare its own log rules (`log_rules`) and metrics (`performance_metrics`) under `behaviors`.

- `log_rules`: a list of rule objects where log severity, triggering conditions, and emitted message formats can be specified.
- `performance_metrics`: an object with a flag that enables the measurement of the duration of the target function (`execution_time`), and the definitions of the Prometheus metrics (Section 2.5.2).

Table 4.5 documents the schema for these sub-policies.

4.4.1.3 Logging Rule Declaration Modes

In DBI, logging formats can vary widely across different inspection environments. During exploratory reverse engineering, an analyst often needs a quick trace of the calls to understand execution control flow. Conversely, during an incident response, logs should be able to capture precise parameter states and outcome codes formatted to match external ingestion tools.

Every log message is built from a *format string*: a text with placeholders in braces, followed by the values that fill them. The `fmt` format library used by Spdlog converts each value to text and writes it, in order, in place of a placeholder. For example, `spdlog::info("opened {}", name)` logs, at the info level, the word `opened` followed by the value of `name`.

To reconcile these different requirements, HookDesigner supports different declaration modes within `behaviors.log_rules`:

1. **Default / Empty Rule (`- {}`):** Injects a function invocation log using default metadata (`name: "rule_1"`, `level: "info"`, `condition: "true"`), only for checking if the target function is called.
2. **Default Rule with Arguments:** The policy author can specify target function parameters to include in the log; the transpiler validates argument identifiers against the target signature and generates a structured, pipe-delimited key-value format string automatically.
3. **Custom Format String:** The author writes the format string. The transpiler parses tokens matching `{placeholder}` (supporting format specifiers like `:x` or `:.2f`) and emits a positional `fmt` format string (Section 4.4.2.4) representing the custom log.
4. **Multi-Conditional Rules:** Emits independent sequential C++ `if` blocks for generating logs evaluated against return codes or runtime arguments.

Code 4.10 shows the modes on an extension of the running example that records the file activity of the program. `NtOpenFile` uses the empty rule. `NtCreateFile` declares two rules

Field	Type	Req.	Default	Description
Performance Metrics Policy (`behaviors.performance_metrics`)				
`execution_time`	bool	No	`false`	Enables `target_function __hook_latency` measurement
`counters`	list	No	`[]`	Counter metric definitions
`gauges`	list	No	`[]`	Gauge metric definitions
`histograms`	list	No	`[]`	Histogram metric definitions
Metric Instrument Schema (Counters, Gauges, Histograms)				
`name`	string	Yes	—	Unique metric identifier within the hook function.
`description`	string	No	`""`	Human-readable help string exported to the Prometheus registry.
`condition`	string	No	`"true"`	Boolean expression indicating when the observation occurs.
`value`	string	Gauges/Hists	—	C++ numerical expression to record (e.g., `__hook_latency`, parameter).
`buckets`	list	No (Hists)	*default*	Custom strictly increasing positive bucket upper bounds only for histograms.
`label`	dict	No	`{}`	Dynamic key-value label mapping for dimensional partitioning.
Logging Rule Schema (`behaviors.log_rules`)				
`name`	string	No	`"rule_N"`	Unique rule name identifier within the hook
`level`	string	No	`"info"`	Log severity level (`trace`, `debug`, `info`, `warn`, `error`, `critical`).
`condition`	string	No	`"true"`	Boolean expression evaluated at runtime to trigger logging.
`arguments`	list	No	`[]`	Parameter identifiers captured for auto-generated format strings.
`format`	string	No	`null`	Custom message format string supporting positional format specifiers.

Table 4.5: Schema definition for hook-level logs and metrics.

(mode 4): the first lists two arguments (mode 2), and the second uses a custom format on failed calls (mode 3); `__hook_result < 0` selects the failures, because a negative `NTSTATUS` is a warning or an error (Section B.2).

Code 4.10: Demonstration of the logging rule declaration modes (file-activity extension of the running example).

```
hooks:
  # Mode 1: Empty Rule
  - module: ntdll.dll
    function_name: NtOpenFile
    behaviors:
      log_rules:
        - {}  # Emits: "[HOOK NtOpenFile] [rule_1] called"

  # Mode 4: Multi-rule
  - module: ntdll.dll
    function_name: NtCreateFile
    behaviors:
      log_rules:
        # Mode 2: Default Rule with Arguments
        - name: "create_args"
          level: "debug"
          arguments:
            - DesiredAccess
            - CreateDisposition
          # Emits: "[HOOK NtCreateFile] [create_args] | DesiredAccess=1179785 |
    CreateDisposition=1"

        # Mode 3: Custom format string with format specifiers (:x)
        - name: "create_failed"
          condition: "__hook_result < 0"
          level: "warn"
          format: "NtCreateFile failed: name={CopyObjectName(ObjectAttributes)} status
    =0x{(ULONG)__hook_result:x}"
          # Emits: "NtCreateFile failed: name=\??\C:\data\a.txt status=0xc0000034"
```

The custom format of the second rule calls `CopyObjectName`, a helper function that the logic file of the running example includes from a small header, to read the file name from `ObjectAttributes`.

Code 4.11 shows the telemetry of another extension of the running example. The hook on `NtOpenFile` denies files as in the running example, sends its log over UDP to a log collector listening on port 1514, and exposes three Prometheus metrics (Section 2.5):

- `opens_total`, a counter of all the calls, with a dynamic label `verdict` that is `"denied"` or `"allowed"`;

- `failed_total`, a counter without labels of the allowed calls that failed, that is, that returned a negative `NTSTATUS` (Section B.2);
- `latency_ms`, a histogram of the duration of the allowed calls. The condition `!__hook_denied` leaves out the denied calls, whose duration is 0 because the original function does not run (Section 4.4.2.2).

Code 4.11: Example YAML with telemetry configuration (metrics extension of the running example).

```yaml
configure:
  process: "filedeny_target.exe"
  logic_file: "logic.cpp"
  telemetry:
    logger: "spdlog"
    logger_config:
      type: "udp"
      host: "127.0.0.1"
      port: 1514
      level: "info"
    metrics: "prometheus"
    metrics_config:
      endpoint: "0.0.0.0:8085"

hooks:
  - module: ntdll.dll
    function_name: NtOpenFile
    address_resolution:
      strategy: export
    behaviors:
      performance_metrics:
        counters:
          - name: "opens_total"
            description: "Calls to NtOpenFile, by verdict of the policy"
            label:
              verdict: '__hook_denied ? "denied" : "allowed"'
          - name: "failed_total"
            description: "Allowed calls to NtOpenFile that failed"
            condition: "!__hook_denied && __hook_result < 0"
        histograms:
          - name: "latency_ms"
            description: "Duration of the allowed calls to NtOpenFile, in milliseconds"
            condition: "!__hook_denied"
            value: "__hook_latency"
            buckets: [0.01, 0.05, 0.1, 0.5, 1.0, 5.0]
      log_rules:
        - name: "denied"
          condition: "__hook_denied"
```

```
        level: "warn"
        format: "denied open: name={CopyObjectName(input_ObjectAttributes)}"
```

4.4.2 Telemetry Expression Translation

The telemetry rules of Section 4.4.1 contain short pieces of C++ code, which this section calls *telemetry expressions*: the `condition` of a log or metric rule, the `value` and the `label` of a metric, and each placeholder of a log `format` (Table 4.5). Before the result of an expression can be computed, different problems must be solved:

1. an expression often reads a parameter of the target function: the hook function must save a copy of this parameter at the right moment within the different phases of its execution (Section 4.3.3), so it must be known which parameters each expression reads and when to read them (Section 4.4.2.1);
2. an expression can read values that are not parameters, such as the return value or the duration of the call, through *wildcards*, such as `__hook_result`: each wildcard must be linked to the C++ code that gives its value (Section 4.4.2.2);
3. a parameter reference or a wildcard can name a value that is not compatible with the instrumentation task or with the target function, for example the return value of a `void` function: such an expression must be rejected (Section 4.4.2.3);
4. a placeholder of a custom log `format` string puts an expression inside a text, for example `access=0x{input_DesiredAccess:x}`: the expressions must be separated from the text (Section 4.4.1.3), and the text must be turned into a format string of the formatting library `fmt` (Section 4.4.2.4).

A hook function could solve these problems at runtime, by keeping the expressions as text, as the policy writes them, and evaluating them on every call of the target function. This approach has two drawbacks:

- solving these problems would add work to every intercepted call, on the thread of the target, against the *High-Performance Telemetry* principle (Section 4.1.1);
- an expression not compatible with the hook would be discovered while the target runs, inside the target process: checking it before the DLL is deployed removes this source of runtime errors.

HookDesigner therefore solves these problems at *transpile time*, before the DLL is built and deployed.

In Stage 2, the transpiler solves the first three problems for each expression with the procedure of Algorithm 1. For the fourth, it scans each custom `format` for its placeholders: it translates the expression of each placeholder with the same procedure, and replaces the placeholder with one that keeps only the format specifier (Section 4.4.2.4).

Algorithm 1 Static telemetry expression validation and translation.

Require: telemetry expression e, hook metadata h
Ensure: translated C++ expression e, updated capture sets and tracking flags on h

for each token `input_<p>` or `output_<p>` in e **do**
  if $p \notin h$.`parameters` **or** (token is `output_` **and** h.`is_noreturn`) **then**
    throw semantic error
  end if
  register p in h.`input_vars_needed` (or h.`output_vars_needed`)
end for
replace every `__hook_tid` in e with `GetCurrentThreadId()`
if `__hook_latency` occurs in e **then**
  if h.`is_noreturn` **then**
    throw semantic error
  end if
  mark h as requiring execution-time tracking
  replace `__hook_latency` with `elapsed_ms_<`h.`function_name>`
end if
if `__hook_result` occurs in e **then**
  if h.`return_type` is `void` **or** h.`is_noreturn` **then**
    throw semantic error
  end if
end if
if `__hook_denied` occurs in e **and** h declares no `on_deny` **then**
  throw semantic error
end if
return e

4.4.2.1 Parameter Capturing

The value of a parameter can change while the hook function runs. The pre-logic and the post-logic can change a parameter that they receive by reference. The original function, instead, can change the memory that a pointer parameter points to. The telemetry code is part of the post-trampoline code and runs after the post-logic (Section 4.3.3), so an expression that names a parameter directly sees only its last value. To let a rule choose

the capturing moment, the policy reads a parameter `<p>` through one of two parameter references:

- `input_<p>`: the value of `<p>` when the hook function starts, *before* the pre-logic runs;
- `output_<p>`: the value of `<p>` after the original function and the post-logic have run.

The hook function saves each value in a local variable with exactly this name, so the transpiler does not change the reference in the expression.

Saving a copy costs time on every call, so the capture is *demand-driven*. The transpiler scans each expression with a regular expression that finds every `input_<p>` and `output_<p>` (lines 1–6 of Algorithm 1), and records `<p>` in the set `input_vars_needed` or `output_vars_needed` of the hook. The hook function captures only the parameters in these sets: a parameter that no expression reads is never copied.

For each parameter in these sets HookDesigner chooses how to safely copy it from its type, which Stage 2 reads from the logic file with libclang (Section 4.2.2.1); Section 5.1.1 describes this choice and the helpers in detail.

4.4.2.2 Telemetry Wildcards

A rule often needs values that are not parameters of the target function and that only the hook function knows, such as the return value or the duration of the call. The policy names them with wildcards, which start with `__hook_` so that they are unlikely to have the same name as a parameter of the target function (lines 7–22 of Algorithm 1):

- `__hook_tid`, the identifier of the thread that made the call: the transpiler replaces it with a call to the Win32 function `GetCurrentThreadId()` (line 7).
- `__hook_latency`, the duration of the original function in milliseconds: the hook function measures it only when `execution_time` is enabled (Table 4.5), so the transpiler enables `execution_time` for the hook and replaces the wildcard with the variable `elapsed_ms_<F>` that holds the measured time (lines 8–14).
- `__hook_result`, the value returned to the caller, and `__hook_denied`, the verdict of call denial (Section 4.2.1.2.1), `true` for a denied call: the hook function declares local variables with exactly these names, so the transpiler keeps them as they are and only checks that they exist in the hook (lines 15–22).

A denied call does not run the original function, so its `__hook_result` is the `return_value` of `on_deny` and its `__hook_latency` is 0: latency metrics on these hooks should use the condition `!__hook_denied`, to keep these zeros out of the distribution (Code 4.11).

4.4.2.3 Validation of Parameters and Wildcards

Each parameter reference and each wildcard becomes a variable of the hook function, or code that reads one. If the hook does not declare that variable, the generated code does not compile, and the author receives a compiler error inside code that they did not write. The transpiler therefore rejects in Stage 2, with a message that names the hook and the wrong name, every expression that uses:

- a parameter reference to a parameter that the target function does not have (lines 1–3 of Algorithm 1);
- `__hook_result` in a hook whose target function returns `void` (lines 15–17);
- `__hook_denied` in a hook that declares no `on_deny`, because only these hooks have a verdict (lines 20–21);
- `output_<p>`, `__hook_latency` or `__hook_result` in a hook whose target function never returns (`is_noreturn`, Section 4.2.1.2): the telemetry code of these hooks runs before the original function (Section 5.1.5.2), so these values never exist (lines 2–3, 9–10 and 16–17).

4.4.2.4 Format String Handling

As introduced in Section 4.4.1.3, a `fmt` format string is a text with placeholders in braces: `fmt` converts the arguments that follow the string to text and writes them, in order, in place of the placeholders.

As illustrated in Figure 4.1 for the log rule of the read-only extension of the running example (Code 4.2), the transpiler scans the custom format for placeholders of the form `{expression[:specifier]}`.

For each placeholder, the transpiler:

1. translates the expression with Algorithm 1 and appends the result to the list of arguments;
2. replaces the placeholder with one that keeps only the specifier, such as `{}` or `{:x}`.

The result is a plain `fmt` format string with its list of arguments.

The template passes this format string to `fmt::format_to` inside the macro `FMT_COMPILE`, which makes the C++ compiler read the string while it builds the DLL and generate the code that writes the message. At runtime, the hook function therefore only converts the

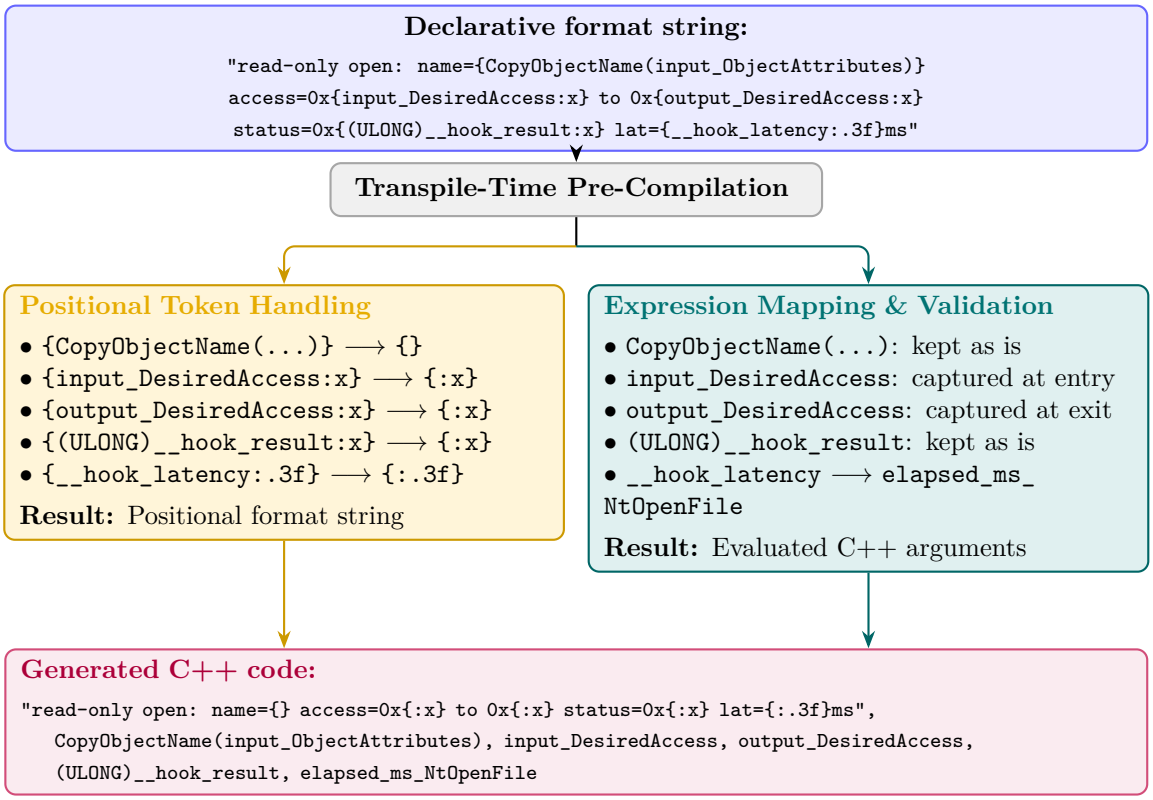


Figure 4.1: Transpile-time format string pre-compilation.

arguments to text. It then passes the finished message to Spdlog as a plain string, not as a format string, so Spdlog does not read it again; passing the format string and the arguments to Spdlog directly would make Spdlog read the string at runtime, each time the rule logs a message.

4.4.3 Asynchronous Logging

As explained in Section 2.5.1, a synchronous logger would add the time of every disk or network write to the intercepted call. When an intercepted function is called often, by several threads at the same time, this wait would be added to every call, would slow down the threads of the target, and would degrade the performance of the software under observation.

4.4.3.1 The Producer-Consumer Scheme

HookDesigner prevents this possible overhead by integrating asynchronous logging powered by Spdlog. During hook initialization in `_telemetry_init.cpp`, the generated code pre-allocates an asynchronous thread pool:

```
spdlog::init_thread_pool(65536, 1);
```

This creates the Spdlog queue (Section 2.5.1) with 65,536 slots instead of the default 8192, and a background thread (the consumer) that empties it, as modeled in Figure 4.2.

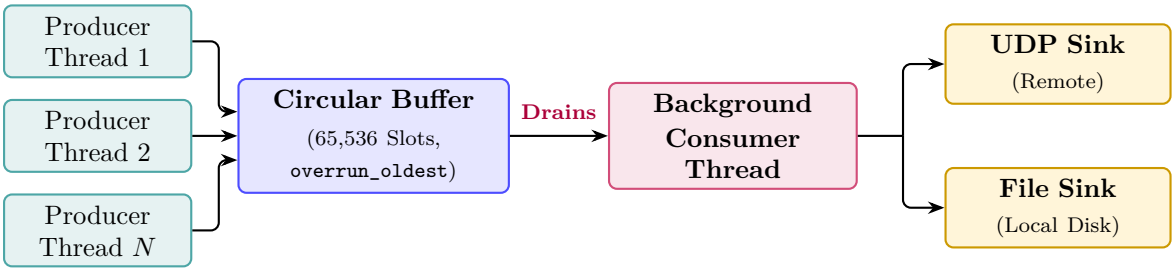


Figure 4.2: Architecture of the asynchronous structured logging subsystem.

When a hook function triggers a log rule, the thread of the target (the producer) only formats the message and copies it into the queue; the disk or network write happens on the background thread.

4.4.3.2 Buffer Saturation Policy

Under burst load, the threads that run the hook functions may produce messages faster than the background thread can write them, and the queue fills up. With the default `block` policy (Section 2.5.1), the thread of the target would then wait for a free slot. With 65,536 (2^{16}) entries, eight times the default of Spdlog, the queue can absorb larger bursts before it fills up. It fills up only if the background thread falls behind the producers for a long time, for example when the disk is very slow. If saturation does occur, HookDesigner avoids the wait by configuring the asynchronous logger with the `overrun_oldest` policy:

```
spdlog::async_overflow_policy::overrun_oldest
```

Under queue saturation, if the 65,536-entry queue fills completely, the logger drops the oldest unwritten messages to make space for incoming events. With this policy, a thread of the target never waits for the disk or the network; it can only wait briefly for the mutex of the queue (Section 2.5.1). This follows the *High-Performance Telemetry* principle established in Section 4.1.1.

4.4.3.3 Network and Storage Sinks

The logging subsystem supports two sinks (Section 2.5.1):

- **UDP Sink (`logger_config.type: "udp"`):** HookDesigner integrates a non-blocking UDP sink using `spdlog::sinks::udp_sink_mt`. Utilizing connectionless UDP transport provides an advantage: if the remote logging server crashes or network connectivity is down, the operating system immediately discards the packets. The payload never blocks attempting to re-establish TCP handshakes, isolating the host process from external network failures.
- **Local File Sink (`logger_config.type: "file"`):** HookDesigner also creates a basic file sink (`spdlog::sinks::basic_file_sink_mt`). The logger writes to disk at once only for error messages (`spdlog::flush_on(spdlog::level::err)`). This keeps the number of disk writes low, and the error messages are still on disk if the target crashes.

4.4.4 Performance Metrics Engine

Logs describe single events, while quantities such as call rates, error ratios and latency distributions are better served by metrics (Section 2.5.2). HookDesigner exposes metrics through `prometheus-cpp`: the hook DLL hosts the embedded HTTP server of the library, from which a Prometheus server scrapes the current values.

4.4.4.1 Registration and Metric Family Hierarchy

During the initialization of the payload (`InitializeHooks`), the generated code allocates a global metric registry and starts an embedded HTTP exposer on the configured socket address, here the endpoint of Code 4.11 (simplified):

```
g_PromRegistry = std::make_shared<prometheus::Registry>();
g_PromExposer = std::make_unique<prometheus::Exposer>("0.0.0.0:8085");
g_PromExposer->RegisterCollectable(g_PromRegistry);
```

For every counter, gauge and histogram (Section 2.5.2) declared under `hook.behaviors.performance_metrics`, HookDesigner generates a typed global pointer to its metric family (`g_PromFamily_<Function>_<Metric>`), created with `BuildCounter`, `BuildGauge` or `BuildHistogram` and registered in the central registry.

4.4.4.2 Dynamic Labels

A HookDesigner metric can declare one dynamic label (Section 2.5.2) whose value is a C++ expression evaluated inside the hook function (Table 4.5), such as the `verdict` label of Code 4.11, which distinguishes denied calls from allowed ones. The value is known only when the hook function runs, so on each observation the hook function must find the time series of that value with `Family::Add`. Each call allocates memory and takes the mutex of the family (Section 2.5.2). When several threads of the program call the target function at the same time, they wait for each other on this mutex, and the wait is added to every intercepted call.

4.4.4.2.1 The Thread-Local Cache HookDesigner avoids this cost with a *Caching strategy* generated into `_telemetry_body.cpp`.

Behind this optimization is *temporal label locality*: within processes, worker threads frequently execute batches of operations that share identical label values over extended time windows (e.g., in the running example, a thread that opens many files in a row, all with the same verdict).

To exploit this locality, the transpiler generates two `thread_local` variables for every labeled metric. In C++, each thread has its own copy of a `thread_local` variable [Mic26p], so every thread keeps its own cache and reads it without locks:

1. `thread_local uint64_t tl_last_hash`: Stores the 64-bit hash of the last observed label value on that specific thread.
2. `thread_local MetricType* tl_last_ptr`: Stores a direct pointer to the underlying Prometheus metric instance.

Code 4.12 illustrates the caching logic generated by the transpiler for the `opens_total` counter of Code 4.11.

Code 4.12: Thread-local caching mechanism.

```
if (g_PromFamily_NtOpenFile_opens_total) {
    // Evaluate label string and compute 64-bit hash
    std::string cur_val = ToLabelString(__hook_denied ? "denied" : "allowed");
    uint64_t cur_hash = std::hash<std::string>{}(cur_val);

    // Thread-local storage variables
    thread_local uint64_t tl_last_hash_opens_total = 0;
    thread_local prometheus::Counter* tl_last_ptr_opens_total = nullptr;

    // Fast-Path Check: bypass mutex if label matches cached state
```

```
    if (tl_last_ptr_opens_total == nullptr || tl_last_hash_opens_total != cur_hash) {
        // SLOW PATH: Mutex acquired only on start or label transition
        tl_last_ptr_opens_total = &g_PromFamily_NtOpenFile_opens_total->Add({ {"verdict
    ", cur_val} });
        tl_last_hash_opens_total = cur_hash;
    }

    // FAST PATH: Lock-free atomic increment
    tl_last_ptr_opens_total->Increment();
}
```

4.4.4.3 Direct Binding for Label-less Metrics

For metrics defined without dynamic labels, HookDesigner binds each metric once, when `InitializeHooks` runs. Because the metric dimensionality is static, resolving the metric instance dynamically at runtime is entirely redundant. During initialization (`_telemetry_init.cpp`), the transpiler generates code that registers the metric family and immediately adds the base instance, storing the result in a static global pointer, as for the `failed_total` counter of Code 4.11:

```
g_PromMetric_NtOpenFile_failed_total = &g_PromFamily_NtOpenFile_failed_total->Add({ {"
    function", "NtOpenFile"} });
```

Inside the hook function (`_telemetry_body.cpp`), the observation bypasses even the thread-local hash check, updating the metric directly through the global pointer:

```
if (g_PromMetric_NtOpenFile_failed_total) {
    if (!__hook_denied && __hook_result < 0) {
        g_PromMetric_NtOpenFile_failed_total->Increment();
    }
}
```

This delivers the best possible execution efficiency, reducing metric recording overhead to a single atomic instruction.

To demonstrate how the different HookDesigner telemetry features operate within the hook function, Code 4.13 illustrates the complete telemetry evaluation block generated by the `_telemetry_body.cpp` template for the policy of Code 4.11.

The block contains Spdlog logging, a cached labeled counter, a label-less counter and a label-less histogram. A gauge, not used by this policy, is generated like a counter, with `Set(value)` instead of `Increment()`.

Code 4.13: Generated telemetry block of the hook function (`_telemetry_body.cpp`) with the telemetry features of this section (metrics extension of the running example).

```
// --- Logging ---
if (__hook_denied) {
    fmt::memory_buffer ss_denied;
    fmt::format_to(std::back_inserter(ss_denied), FMT_COMPILE("denied open: name={} tid
    ={}"),
                   CopyObjectName(input_ObjectAttributes), GetCurrentThreadId());
    spdlog::warn(spdlog::string_view_t(ss_denied.data(), ss_denied.size()));
}

// --- Prometheus Metrics ---
// 2. Custom Counters
if (g_PromFamily_NtOpenFile_opens_total) {
    std::string cur_val = ToLabelString(__hook_denied ? "denied" : "allowed");
    uint64_t cur_hash = std::hash<std::string>{}(cur_val);
    thread_local uint64_t tl_last_hash_opens_total = 0;
    thread_local prometheus::Counter* tl_last_ptr_opens_total = nullptr;

    if (tl_last_ptr_opens_total == nullptr || tl_last_hash_opens_total != cur_hash) {
        tl_last_ptr_opens_total = &g_PromFamily_NtOpenFile_opens_total->Add({ {"verdict
    ", cur_val} });
        tl_last_hash_opens_total = cur_hash;
    }
    tl_last_ptr_opens_total->Increment();
}
if (g_PromMetric_NtOpenFile_failed_total) {
    if (!__hook_denied && __hook_result < 0) {
        g_PromMetric_NtOpenFile_failed_total->Increment();
    }
}

// 3. Custom Gauges

// 4. Custom Histograms
if (g_PromMetric_NtOpenFile_latency_ms) {
    if (!__hook_denied) {
        g_PromMetric_NtOpenFile_latency_ms->Observe((double)(elapsed_ms_NtOpenFile));
    }
}
```

4.5 Summary

This chapter presented how HookDesigner generates a hook DLL from an instrumentation DSL policy. HookDesigner follows a pipeline of seven stages: the first four run offline on the developer's machine, and only the compiled DLL and the injector are needed on

the target host. The user writes a YAML policy, which says which functions to hook and which behaviors to attach to them, and a C++ logic file, which holds the signatures of these functions and the code of their pre-logic and post-logic. A pre-logic can also deny a call: the original function then does not run, and the caller receives the value declared in `on_deny`. An engine registry and a two-layer hierarchy of Jinja2 templates keep the engine-specific code apart from the code that the engines share, so the same policy generates hooks for Microsoft Detours and for Frida-Gum, together with the lifecycle functions `InitializeHooks` and `DeinitializeHooks`. The telemetry rules of the policy are translated into C++ when the code is generated, instead of being interpreted at runtime. Logs go through the asynchronous logger of Spdlog, and metrics through a Prometheus HTTP endpoint inside the DLL. The generated DLL runs inside the target process, so a fault in its code can crash the target: Chapter 5 shows how HookDesigner prevents this, and how the DLL is safely injected and ejected.

Chapter 5

Runtime Safety and DLL Lifecycle

Chapter 4 showed how HookDesigner turns a policy and a logic file into a self-contained hook DLL. This chapter follows that DLL into the target process, where it runs during the online stages 5–7 of the pipeline (Section 4.1.2). Section 5.1 shows the safety measures adopted to avoid possible crashes in the target process, following the *Crash Safety* design principle (Section 4.1.1), and ends with the complete structure of a generated hook function. Section 5.2 then shows how the injector loads the DLL and activates its hooks, and how the DLL is removed while threads may still be inside it, following the *Thread Safety* principle. As in Chapter 4, the examples come from the running example (Section 1.2).

5.1 DLL Runtime Safety

As established in Section 2.1.1, during DBI, the compiled payload runs directly inside the target application's address space without memory isolation boundaries. While this shared-memory model enables DBI, it introduces several runtime safety issues: a fault or unhandled exception anywhere within the injected DLL—whether during lifecycle orchestration (`InitializeHooks`, `DeinitializeHooks`) or inside hook function execution (`_hook_body.cpp`)—immediately crashes the host process. Such a crash violates the *Crash Safety* design principle established in Section 4.1.1.

To satisfy this requirement, HookDesigner adds four protection mechanisms. Three of them run inside every hook function: helpers that read the parameters safely (Section 5.1.1), a guard that stops re-entrant calls (Section 5.1.2), and a guard that keeps the error state of the thread, so that the hook function does not change what the caller sees (Section 5.1.3). The fourth, exception boundaries in both the hook functions and the lifecycle routines, contains the faults that still happen (Section 5.1.4). Section 5.1.5 then shows how these

mechanisms are combined in one hook function.

5.1.1 Transpiler Type Analysis and Memory Access Helpers

As already explained in Section 4.4.2.1, DBI authors often need to inspect function parameters on function entry and exit. However, manually writing C++ code to copy the value of a parameter is error-prone: forgetting a null check, dereferencing an invalid pointer, or traversing an unbounded string can easily cause memory access violations. To help the work of policy authors, HookDesigner *automates* access to primitive variables and strings in a safe way. By analyzing parameter types at transpile time, HookDesigner automatically generates the appropriate access solution, preventing the author from writing repetitive and error-prone C++ code.

5.1.1.1 Parameter Type Analysis

On each parameter requested by a telemetry expression (Section 4.4.2.1), the transpiler performs automatic type analysis on the Clang type representation (preserved from Stage 2, Section 4.2.2.1).

This automated classification enables the transpiler to emit uniform assignments of the form `safe_type input_<p> = safe_read_expr` without requiring template branching in the hook body. Table 5.1 formalizes the complete type mapping matrix.

- **Pointer Indirection Resolution:** By iterating on the pointee types, the transpiler calculates the pointer depth d.
- **Safe Access Expression:** For dereferenceable primitive pointers, it nests d dereference helpers (`ReadPointer`). Opaque handles (alias for `void*`, C++ does not allow dereferencing a `void*`).

5.1.1.2 Memory-Safety Helper Functions

Most of the generated expressions leverage helper functions defined in `_user_logic_helpers.cpp`:

- **Pointer Dereferencing (`ReadPointer<T>`):** When reading data through a pointer, standard C++ dereferencing (`*ptr`) crashes the process if the pointer is null or unmapped. HookDesigner defines a templated helper:

Clang Type	Depth (d)	Category	Generated Read Expression	Safe Type
`int`, `double`, `DWORD`	0	Primitive Value	`param`	`T` (original type)
`const char*`, `char*`	1	C-String	`CopyString(param)`	`std::string`
`const wchar_t*`, `wchar_t*`	1	Wide C-String	`CopyString(param)`	`std::string` (UTF-8)
`PUNICODE_STRING`	1	Counted NT String	`CopyUnicodeString(param)`	`std::string` (UTF-8)
`double*`	1	Primitive Pointer	`ReadPointer(param)`	Underlying type (`double`)
`int**`	2	Nested Pointer	`ReadPointer(ReadPointer(param))`	Underlying type (`int`)
`void*`, `HANDLE`	1	Opaque Handle	`param`	Raw pointer type

Table 5.1: Generated read expression for each Clang type.

```
template <typename T>
static T ReadPointer(const T* ptr, T fallback = T()) {
    if (!ptr) {
        return fallback;
    }
    return *ptr;
}
```

If `ptr` contains a non-null but unmapped address (e.g., a dangling pointer pointing to deallocated memory), the read still faults; this fault is caught by the exception boundary that surrounds the parameter capture (Section 5.1.4.1).

- **Bounded String Copying (`CopyString`):** Passing raw string pointers (`char*` or `wchar_t*`) directly to logging formatters is dangerous: if the target buffer lacks a null terminator or points to invalid memory, formatting routines read past the buffer boundary until they fault. HookDesigner generates explicit calls to `CopyString`:

```
static std::string CopyString(const char* source, size_t maxChars = 512) {
    if (!source) return "null";
```

```
    size_t len = strnlen(source, maxChars);
    return std::string(source, len);
}
```

The bound of 512 characters covers Windows file paths (`MAX_PATH` is 260 characters) and the longest registry key paths, while it still limits the scan of a corrupted or non-terminated buffer. For wide strings (`wchar_t*`), an overloaded implementation is used.

- **Counted String Copying (`CopyUnicodeString`):** Functions of the Windows native API (`ntdll.dll`) pass names as `UNICODE_STRING` structures, as presented in Section B.2. `CopyString` cannot be used here, because it searches for a terminator that may not exist.

5.1.1.3 Member Access Contract

When a telemetry expression references an internal struct member using the arrow or dot operator (e.g., `{input_ObjectAttributes->RootDirectory}`), the transpiler captures only the parameter and does not automatically analyze the type of the field. Instead, it emits the member access expression defined in the YAML policy directly into the generated C++ code.

This design establishes an explicit contract:

- **Author Flexibility:** The policy author can read nested member fields: HookDesigner accepts such expressions.
- **Author Responsibility:** The author is responsible for reading pointer and string members safely, with the helpers of `_user_logic_helpers.cpp`. In the running example, the `ObjectName` field of the `OBJECT_ATTRIBUTES` structure is a counted string reached through a struct, so the author wraps it as `CopyUnicodeString( input_ObjectAttributes->ObjectName)`. The logic file of the running example packs this check in two small helpers, `CopyObjectName` and `ObjectRoot`, which also return a safe value when `ObjectAttributes` is `NULL`; the log rules call them instead of the raw member access (Code 4.10).
- **Exception Boundary Safety:** If the author makes a mistake, the resulting fault is caught by the exception boundary around the telemetry code (Section 5.1.4.1), so the host process does not crash.

5.1.2 Re-entrancy Guard

A hook function that calls a target function, directly or through another target function, enters a hook function again on the same thread, and can loop until the stack overflows (Section 2.3.1). Detours does nothing against these re-entrant calls, and Frida-Gum stops only the direct form, and only with Standard Replacement (Section 3.3.1). This section explains why HookDesigner adds its own guard to the hook functions of both engines, and how the guard works.

5.1.2.1 A Guard for Both Engines

With Detours, the hook functions need a guard, because the engine has no check of its own. With Frida-Gum, the built-in check of the dispatcher is not enough either, for two reasons. First, the calls in the indirect form still reach the hook functions, and they do harm even when they do not loop (Section 3.3.1). Second, for functions that never return, HookDesigner installs the Frida-Gum hook with `gum_interceptor_replace_fast` (Section 5.1.5.2); this mode does not use the dispatcher (Section 3.2.1), so no check runs at all.

For these reasons, HookDesigner generates the same guard in the hook functions of both engines: a flag for each thread, shared by all the hooks of the DLL. While any hook function runs, the flag is set, and a call to any target function on the same thread goes straight to the original function. The flag is set at every entry of a hook function, with both engines, because a hook function cannot know in advance whether its logic will call a target function. What changes between the engines is the mechanism that stops each re-entrant call:

- With Detours, and with Frida-Gum's Fast Replacement, every re-entrant call reaches the hook function, and the guard of HookDesigner stops both forms.
- With Frida-Gum's Standard Replacement, a call in the direct form never reaches the hook function, because the dispatcher sends it to the original function first: Frida-Gum's built-in mechanism stops it, and the guard of HookDesigner never sees it. Only the calls in the indirect form reach the hook function, where the guard of HookDesigner stops them.

5.1.2.2 The ReEntrancyHookGuard Definition

HookDesigner stores the flag of the guard in a `thread_local` variable.

```
static thread_local bool t_InHook = false;
```

Using this flag, HookDesigner implements a helper structure named `ReEntrancyHookGuard` in the helper file `_user_logic_helpers.cpp`, shown in Code 5.1. The structure follows the C++ resource acquisition is initialization (RAII) idiom: a local object takes a resource in its constructor and gives it back in its destructor. C++ runs the destructor automatically when the object goes out of scope, both when the function returns and when an exception leaves the function, so the resource is given back on every possible function exit path.

Code 5.1: `ReEntrancyHookGuard` RAII implementation in `_user_logic_helpers.cpp`.

```
struct ReEntrancyHookGuard {
    bool active;
    ReEntrancyHookGuard() : active(false) { Acquire(); }
    ~ReEntrancyHookGuard() { Release(); }

    void Acquire() {
        if (!active) {
            t_InHook = true;
            active = true;
        }
    }

    void Release() {
        if (active) {
            t_InHook = false;
            active = false;
        }
    }
};
```

5.1.2.3 The New Hook Function Control Flow

At the entry of each generated hook function (`Hooked<F>`), before executing any code, the function reads the flag of the thread:

```
if (t_InHook) {
    return True<F>(param1, param2, ...);
}
```

If the flag is set, the thread is already executing inside an active hook function. The function immediately redirects execution to the trampoline pointer (`True<F>`), without running the pre-logic, the post-logic or the telemetry code.

If the flag is not set, the function instantiates a local `ReEntrancyHookGuard` on the stack, which sets the flag (`Acquire()`). Crucially, right before invoking the trampoline to execute the original target function, the function calls:

```
hook_reEntrancy_guard.Release();
```

This pre-trampoline release is a deliberate choice:

- While the pre-trampoline and post-trampoline code run, the guard is *active* (`true`), blocking any re-entrant hook loops triggered by the pre-logic, the post-logic or the telemetry code.
- While the original target function executes, the guard is *released* (`false`). If the target function legitimately calls *another* target function as part of its normal operation, that nested call is permitted to enter its own hook function and be monitored.

Under Frida-Gum's Standard Replacement, this release has no effect on a call to the *same* function made by the original function: the dispatcher skips it before the hook function runs (Section 3.3.1), so a recursive target function can be correctly monitored with Detours but not with Frida-Gum.

After the trampoline returns, the function re-acquires the guard (`hook_reEntrancy_guard.Acquire()`) to protect the rest of the post-trampoline code: the post-logic and the telemetry code.

Figure 5.1 in Section 5.1.5 shows these steps within the complete control flow of a generated hook function.

5.1.3 System Error State Preservation

As discussed in Section 2.2, the hook function must leave the machine state of the thread as the target function would leave it. A thread, however, also carries a *software* error state that the hook function must preserve: the Win32 *last-error code*. Windows keeps this value per thread, so all modules of the process read and write the same value [Mic26h]. A caller typically reads it with `GetLastError()` right after a failed call, to learn the reason of the failure.

Inside a hook function, a lot of code runs between the entry and the return to the caller, and much of it changes the last-error code as a side effect. Without protection, the caller would read the error left by the hook function instead of the one produced by the original function, and could take a different execution path: the hook function would change the behavior of the program it observes.

HookDesigner prevents this with `SystemErrorGuard`, a RAII helper defined in `_user_logic_helpers.cpp` and shown in Code 5.2.

Code 5.2: `SystemErrorGuard` in `_user_logic_helpers.cpp` (simplified).

```
struct SystemErrorGuard {
    DWORD win32_error;   // errno is saved and restored in the same way

    SystemErrorGuard() : win32_error(GetLastError()) {}
    ~SystemErrorGuard() { SetLastError(win32_error); }

    void Capture() { win32_error = GetLastError(); }
    void Restore() const { SetLastError(win32_error); }
};
```

Each generated hook function uses the guard at four points:

1. **Entry:** the guard is the first object constructed after the re-entrancy check, so it saves the caller's last-error code before any other code of the hook function runs. The check itself only reads the `thread_local` flag (Section 5.1.2), which does not change the error state.

2. **Before the original function:** `Restore()` is called right before the call to `True<F>`. The original function therefore starts from the same error state it would see without the hook. The re-entrant bypass path does not use the guard: it runs only the check and the call to `True<F>`, so the original function starts from the caller's error state, and the caller sees the one left by the original function.

3. **After the original function:** `Capture()` is called once `True<F>` returns, and saves the error state produced by the original function. When execution time is measured, `Capture()` follows the end timestamp, just as `Restore()` precedes the start timestamp, so the timed window contains only the original function. `QueryPerformanceCounter` modifies neither the last-error code nor `errno`, so the saved state is the same.

4. **Return:** the destructor writes the saved value back on every return path that follows the check.

As a consequence, the post-logic, the output capture and the telemetry code can change the last-error code freely: the caller always observes the value produced by the original function.

The call-denial branch (Section 4.2.1.2.1) relies on the same mechanism: when `last_error` is declared, it writes that value into the guard, and the destructor returns it to the caller.

Figure 5.1 in Section 5.1.5 shows these steps within the complete control flow of a generated hook function.

5.1.4 Asynchronous Exception Handling

The helpers of Section 5.1.1 avoid the most common faults, but not all of them: a non-null pointer to unmapped memory still faults, and the code of the logic file, the telemetry code and the lifecycle functions can also fault or throw a C++ exception. This section shows how HookDesigner contains these faults, so that they do not reach and make the host process crash.

In standard C++ development, Windows compilers typically use the synchronous exception model (`/EHsc`) [Mic26c]. Under `/EHsc`, C++ `try/catch` blocks are designed only to intercept exceptions thrown explicitly through the C++ `throw` statement. Under this standard configuration, a C++ `catch(...)` block does not catch OS-level faults—such as dereferencing a null pointer, reading an unmapped virtual address, or encountering a division by zero. In this way, the exception bypasses user code, triggers the default operating system exception filter, and terminates the host process immediately.

To have a unique exception block and so keep the generated code simpler, HookDesigner wants a single `catch(...)` block to catch both kinds of errors: C++ exceptions raised by `throw` and OS-level faults. Without this, every protected region would also need the Microsoft-specific `__try/__except` construct of structured exception handling (SEH). For this reason, the transpiler compiles the hook DLL with the asynchronous exception model [Mic26c], passing the `/EHa` flag to the Microsoft Visual C++ (MSVC) compiler.

5.1.4.1 Exception Boundaries

To prevent cascading failures and isolate runtime faults across the DLL, HookDesigner organizes exception boundaries across two tiers:

5.1.4.1.1 Hook Lifecycle Functions Tier: The exported lifecycle routines `InitializeHooks` and `DeinitializeHooks` are enclosed in `try/catch` blocks compiled under `/EHa`. While the template listings in (Section 4.3.4) focused on the primary attachment and detachment workflows, production safety requires explicit transaction rollback and resource cleanup inside these exception handlers, as shown in Code 5.3.

If an exception or hardware fault occurs during target address resolution, transaction commits or telemetry initialization, the fault is safely intercepted, logged via `OutputDebugStringA`, and mapped to an explicit failure exit code.

Crucially, if a fault occurs in `InitializeHooks` while an engine transaction is open (`in_transaction == true`), the exception handler must make sure that no staged hook is written to memory, so that the instrumentation is never partial (Section 3.5):

- **Microsoft Detours:** If a `DetourAttach` call fails, Detours rolls back the transaction by itself: `DetourTransactionCommit` sees the error and aborts the whole transaction. A fault before the commit, instead, skips the commit, so the handler calls `DetourTransactionAbort()` explicitly. This call discards all staged hooks and resumes the threads suspended by `DetourUpdateThread` (Section 3.5).
- **Frida-Gum:** Frida-Gum has no abort primitive, and `gum_interceptor_end_transaction()` writes all staged hooks to memory. For this reason, the `std::exception` handler (which also receives a failed replacement, Section 4.3.4.1) first disarms the staged hooks with `gum_interceptor_revert()` and only then closes the transaction with `gum_interceptor_end_transaction()`. It reverts only the hooks whose trampoline pointer `True<F>` is set, because Frida-Gum writes this pointer only when a replacement succeeds.

Finally, the telemetry structures already built are destroyed. In this way, after a failure during initialization, no staged hook and no telemetry resources remain, and the injector can safely unload the DLL (Section 5.2).

The handlers of `DeinitializeHooks` are simpler, and they differ between the two engines because of what an open transaction holds. Under Detours, `DetourUpdateThread` has already suspended the other threads of the process (Section 3.5). If a fault interrupts the transaction before the commit, the handler calls `DetourTransactionAbort()`, which resumes these threads; without it, the host process would stay frozen. The commit never ran, so the original prologue bytes were not written back and the hooks stay attached. Under Frida-Gum, there is nothing to undo: before `gum_interceptor_end_transaction()`, the transaction suspends no thread and changes no code, because `gum_interceptor_revert()` on a hooked target only updates the interceptor's own tables. For this reason, its handlers contain no rollback code.

Unlike the handlers of `InitializeHooks`, they do not destroy the telemetry structures; under Frida-Gum, they also keep the interceptor. After a fault, `DeinitializeHooks` cannot tell whether the hooks are still attached (the fault hit the detach transaction) or whether threads are still running inside the hook functions. In both cases a thread may still run the hook code, which writes to the logger and the metrics. If the telemetry structures were destroyed, that thread would use freed memory and crash the host process. The only telemetry part that is already gone is the Prometheus HTTP exposer (Section 4.4.4), because `DeinitializeHooks` stops it before it detaches the hooks.

Finally, the handlers log the error and return a non-zero code. The injector checks the value returned by `DeinitializeHooks` and, when it is non-zero, does not unload the DLL. The reason is the same as for the telemetry structures: a thread may still run the hook code, and if the DLL were unloaded, that thread would run code at unmapped addresses and crash the host process (Section 2.3.1). The injector uses the same fallback when threads

stay blocked inside the hook functions (Section 5.2.3.3).

Code 5.3: Atomic transaction rollback and resource containment in lifecycle exception handlers.

```
// catch blocks inside InitializeHooks
catch (const std::exception& ex) {
    // Engine Transaction Rollback
    // Detours:
    if (in_transaction) {
        DetourTransactionAbort();
        for (HANDLE hThread : threads) {
            CloseHandle(hThread);
        }
    }

    // Frida-Gum (explicit revert loop before concluding transaction):
    if (in_transaction && interceptor) {
        {% for hook in hooks %}
        // True<F> is set only by a successful replace
        if (True{{ hook.function_name }}) {
            gum_interceptor_revert(interceptor, target_{{ hook.function_name }});
        }
        {% endfor %}
        gum_interceptor_end_transaction(interceptor);
    }
    if (interceptor) {
        g_object_unref(interceptor);
        interceptor = nullptr;
        gum_deinit_embedded();
    }

    // Teardown initialized telemetry
    g_PromExposer.reset();
    g_PromRegistry.reset();
    spdlog::shutdown();

    OutputDebugStringA(msg);
    return 0xE0000002; // Custom failure code for C++ exception returned to injector
}
catch (...) {
    // Same as above
    OutputDebugStringA("[CRITICAL] SEH/Hardware fault in InitializeHooks\n");
    return 0xE0000003; // Custom failure code for SEH/Hardware fault returned to
    injector
}
```

5.1.4.1.2 Hook Function Tier: Inside the body of each hook function (`_hook_body.cpp`), execution is divided into decoupled, independent exception boundaries, as shown in Code 5.4:

1. **Parameter Capture Boundaries:** Entry and exit argument captures are wrapped in dedicated exception boundaries. If an SEH hardware fault occurs, the error is logged with the corresponding parameter name, and execution safely continues without skipping the pre-logic or the post-logic.
2. **Pre-Logic and Post-Logic Boundaries:** The pre-logic and the post-logic execute inside separate, independent exception blocks. A fault within user code is contained, guaranteeing that the target function is still invoked via its trampoline (`True<F>`), that the telemetry code is executed and that the return value is safely passed back to the caller. The only exception is a hook that declares `on_deny`: there, a fault in the pre-logic denies the call instead (fail-closed, Section 4.2.1.2.1 and Section 5.1.5.3).
3. **Telemetry Evaluation Boundary:** A fault in the telemetry code is caught, and the hook function still returns `__hook_result` to the caller.

5.1.5 The Complete Hook Function Structure

All the presented safety features converge in the body of the generated hook function (`_hook_body.cpp`). This body varies on two levels:

- **At code generation:** the transpiler emits either the standard body or the body for non-returning functions (Section 5.1.5.2), and it emits the call-denial branch (Section 5.1.5.3) only if the hook declares `on_deny` (Section 4.2.1.2.1). A given hook function therefore contains only one of these variants.
- **At runtime:** the generated code takes only two decisions: the re-entrancy check (Section 5.1.2) and, if present, the deny-call verdict.

Figure 5.1 shows all the resulting paths in a single flowchart. The subsections that follow give the complete generated code of the standard body, and then describe each variant on its branch of the flowchart.

5.1.5.1 Standard Hook Function Body

To illustrate how these defenses integrate in practice for standard returning targets, Code 5.4 presents a complete generated C++ hook function; it follows the allowed-call

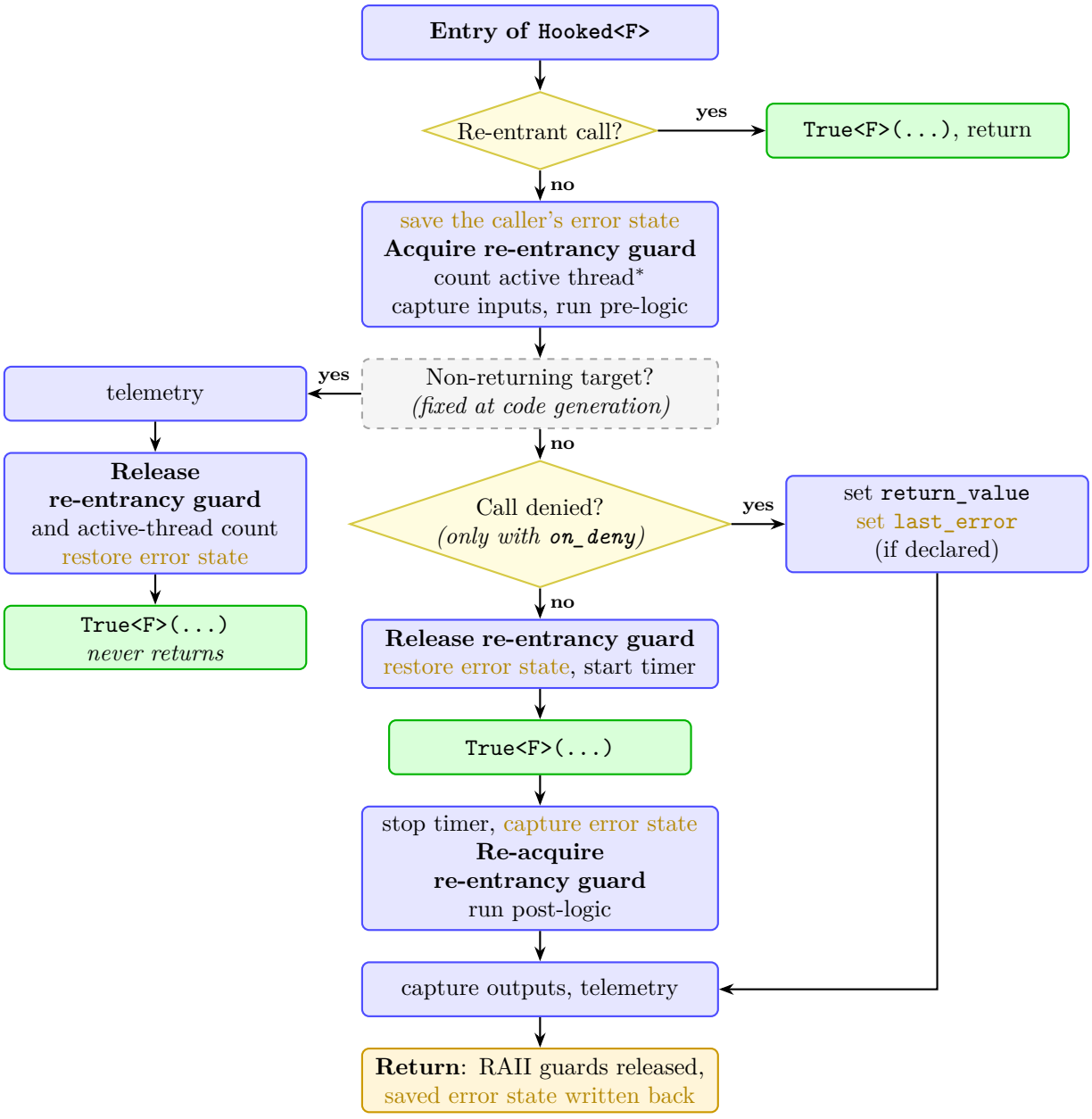


Figure 5.1: Control flow of a generated hook function. *Only in the hook functions that count the active threads: all the Detours ones, and the Frida-Gum ones for non-returning functions (Section 5.2.3.1).

path in the middle of Figure 5.1. It is the hook function for `NtOpenFile` of the read-only extension (Code 4.2). Right after the re-entrancy guard, the function also declares `hook_active_threads`, another RAII helper that counts the threads running inside the hook functions, so that the DLL is not unloaded while one of them is still running (Section 2.3.1); Section 5.2.3.1 describes this helper.

Code 5.4: Complete generated hook function demonstrating integrated safety mechanisms and decoupled exception boundaries (`NtOpenFile`, read-only extension of the running example).

```
NTSTATUS NTAPI HookedNtOpenFile(PHANDLE FileHandle, ACCESS_MASK DesiredAccess,
        POBJECT_ATTRIBUTES ObjectAttributes, PIO_STATUS_BLOCK IoStatusBlock,
        ULONG ShareAccess, ULONG OpenOptions) {
    if (t_InHook) {
        return TrueNtOpenFile(FileHandle,DesiredAccess,ObjectAttributes,IoStatusBlock,
    ShareAccess,OpenOptions);
    }

    SystemErrorGuard system_error_guard;
    ReEntrancyHookGuard hook_reEntrancy_guard;
    HookActiveThreads hook_active_threads;

    bool telemetry_inputs_ready = false;
    ACCESS_MASK input_DesiredAccess = {};
    POBJECT_ATTRIBUTES input_ObjectAttributes = {};

    try {
        input_DesiredAccess = DesiredAccess;
        input_ObjectAttributes = ObjectAttributes;
    } catch (const std::exception& ex) {
        OutputDebugStringA(msg);
    } catch (...) {
        OutputDebugStringA(msg);
    }

    // Pre-logic execution
    try {
        ForceReadOnly(DesiredAccess,ObjectAttributes);
    } catch (const std::exception& ex) {
        OutputDebugStringA(msg);
    } catch (...) {
        OutputDebugStringA("[HOOK EXCEPTION] Hardware/SEH fault in pre-hook NtOpenFile\
    n");
    }

    hook_reEntrancy_guard.Release();
    system_error_guard.Restore();

    NTSTATUS __hook_result = TrueNtOpenFile(FileHandle,DesiredAccess,ObjectAttributes,
```

```
    IoStatusBlock,ShareAccess,OpenOptions);

    system_error_guard.Capture();
    hook_reEntrancy_guard.Acquire();

    ACCESS_MASK output_DesiredAccess = {};

    try {
        output_DesiredAccess = DesiredAccess;
    } catch (const std::exception& ex) {
        OutputDebugStringA(msg);
    } catch (...) {
        OutputDebugStringA(msg);
    }

    try {
        // --- Logging ---
        if (input_DesiredAccess != output_DesiredAccess) {
            // ...fmt formatting
            spdlog::info(spdlog::string_view_t(ss_readonly.data(), ss_readonly.size()))
    ;
        }
    } catch (const std::exception& ex) {
        OutputDebugStringA(msg);
    } catch (...) {
        OutputDebugStringA("[TELEMETRY EXCEPTION] Hardware/SEH fault in telemetry
    NtOpenFile\n");
    }

    return __hook_result;
}
```

5.1.5.2 Adaptation for Non-Returning Targets

The standard hook function body operates on the premise that the intercepted target function eventually returns after execution. However, some system APIs are *non-returning functions*: they terminate the thread or process, or transfer control directly to an external exception dispatcher, never returning control to the caller.

If a non-returning function is instrumented using the standard returning function structure, the instrumentation payload encounters the following architectural failures:

1. **Telemetry after the trampoline never runs:** In `_hook_body.cpp`, telemetry evaluation is part of the post-trampoline code, so it is never reached with this type of function.

2. **Bypassed C++ Stack Unwinding:** When the target function ends the thread or the process (`ExitThread`, `ExitProcess`), the operating system immediately tears down the user-mode execution context. Standard C++ stack unwinding is skipped, meaning that local RAII destructors on the stack are *never executed*. The destructor of `hook_active_threads` would therefore never lower the count of threads inside the hook functions, so the ejection would wait for a thread that no longer exists, and the DLL could not be unloaded (Section 5.2.3).

HookDesigner resolves these issues through automated adaptations applied to the standard hook function body:

1. **Pre-Trampoline Telemetry:** In `_hook_body.cpp`, telemetry evaluation is moved into the pre-trampoline code, so it runs *before* the trampoline call.
2. **Early Release of the Thread Count:** The hook function calls `hook_active_threads.Release()` just before it calls the original function, so the count goes down while the thread is still alive (Section 5.2.3.1).

These adaptations form the left branch of Figure 5.1. Before the terminal call, the function also restores the caller's error state (Section 5.1.3); no `Capture()` follows, since the original function never returns to the hook function.

5.1.5.3 Adaptation for Call Denial

When a hook declares `on_deny` (Section 4.2.1.2.1), the transpiler adds a runtime branch to the standard body, shown on the right of Figure 5.1. The flowchart does not show how the verdict is computed. Code 5.5 shows this part of the generated code for the native `NtOpenFile` function of Code 4.4.

Code 5.5: Generated code that computes the call-denial verdict for `NtOpenFile` of the extension in Code 4.4 (simplified).

```
// In HookedNtOpenFile, after the entry, the guards and the input capture of the
    standard body
bool __hook_denied = true;              // fail-closed default
try {
    __hook_denied = !CheckFileAllowed(ObjectAttributes);
} catch (...) {                         // C++ exceptions and SEH faults
    OutputDebugStringA("[HOOK EXCEPTION] Hardware/SEH fault in pre-hook NtOpenFile (
    call denied)\n");
}
// __hook_denied selects the denied path or the allowed path of the flowchart
```

The branch relies on the following design choices:

- **Fail-closed default:** `__hook_denied` starts as `true` and is set only by a pre-logic that returns normally, so a pre-logic that throws or faults leaves the call denied.
- **Denied path:** it only sets `__hook_result` to the declared `return_value` and, when `last_error` is declared, writes it into the error guard (`system_error_guard.win32_error`), whose destructor returns it to the caller (Section 5.1.3). It skips the trampoline call and the post-logic, and then joins the rest of the post-trampoline code: the output capture and the telemetry code.
- **Re-entrancy guard:** on the denied path the guard is never released, because there is no original call to release it around, so the whole hook function runs protected from re-entrancy.
- **Post-logic:** its call is generated inside the allowed path, since it processes the result of an original call.

5.2 DLL Lifecycle and Deployment

After the offline compilation stage (Stage 4, Section 4.1.2) completes, the instrumentation artifact exists as a self-contained Windows DLL. To monitor and manipulate the host application, this payload must be moved to the online context: it must be mapped into the target process's virtual address space and activate its hooks (Stage 5), execute safely alongside the application (Stage 6), and cleanly detach when monitoring terminates (Stage 7).

This section examines the deployment and runtime lifecycle of HookDesigner payloads, managed by the standalone injector utility (`injector.exe`, implemented in `injector.cpp`).

5.2.1 Injection into a Running Process

In modern operating systems, address space isolation (Section 2.1.1) prevents one process from directly manipulating the execution state of another. To introduce the compiled payload into a running target, HookDesigner implements dynamic library injection via the standard Windows debugging and threading APIs.

5.2.1.1 The Two-Phase Protocol

As explained in Section 4.3.4, HookDesigner generates the exported routine `InitializeHooks` for attaching the hooks from outside the `DllMain`, preventing possible Loader Lock issues. For this reason, HookDesigner separates DLL mapping from hooks activation using a *Two-Phase Protocol*:

- **Phase 1 (Passive Mapping):** To attach to a running target process identified by its PID, the standalone injector utility (`injector.exe`) leverages the classical remote thread injection technique introduced in Section 2.1.2. Once `LoadLibraryA` returns, the hook DLL is loaded in the target process, but its hooks are still disabled.
- **Phase 2 (Active Initialization):** Once the Phase 1 remote thread exits, the injector finds the address of `InitializeHooks` in the target and runs it in a second remote thread. Because this second thread runs entirely outside `DllMain` and so outside Loader Lock, it can safely initialize the Spdlog background thread pool, bind the Prometheus HTTP metrics server, and install transactional inline hooks, bypassing the Loader Lock deadlock risk.

Furthermore, the injector inspects the remote exit code returned by `InitializeHooks`. In injection mode, if initialization fails, the injector automatically invokes `FreeLibrary` to unload the DLL, so that no unused DLL stays in the target.

5.2.1.2 Engine-Specific Initialization Mechanics

While Section 4.3.4 introduced the simplified template structure used to emit the `InitializeHooks` skeleton (Codes 4.8 and 4.9), HookDesigner generates a more complex initialization process during Phase 2. This process incorporates the features of Sections 4.4 and 5.1, adapting the attachment strategy to the underlying engine as summarized in Table 5.2.

5.2.2 Injector Launch Mode

Injection mode attaches to a process that is already running. However, this approach introduces a problem: any operations performed by the target before the injector attaches are missed. To solve this problem, `injector.cpp` also provides a *launch mode* designed for directly launching a process already containing the instrumentation payload.

Initialization Step	Microsoft Detours	Frida-Gum
Latency Timer	`QueryPerformanceFrequency()`, only if `has_execution_time` (Section 4.3.2.2)	Identical
Telemetry Engine	Initialized via `_telemetry_init.cpp` (Spdlog thread pool, Prometheus Exposer)	Identical
Runtime Core	None (Direct binary patching)	Calls `gum_init_embedded()` and `gum_interceptor_obtain()`
Address Resolution	Evaluates `GetProcAddress` or base + RVA offset	Identical
Thread Enumeration	Calls `EnlistAllThreads()` via Toolhelp32 snapshot	None
Transactional Scope	`DetourTransactionBegin()`	`gum_interceptor_begin_transaction()`
Thread Enlistment	Calls `DetourUpdateThread()` for all the other threads of the target	None
Hook Attachment	`DetourAttach( &(PVOID&)True<F>, Hooked<F>)`	Calls `gum_interceptor_replace()` (standard) or `replace_fast()` (non-returning)
Transaction Commit	`DetourTransactionCommit()` (suspends threads, relocates IP, applies patch)	`gum_interceptor_end_transaction()` (writes the patch while threads run, no IP relocation)

Table 5.2: Comparison of hook initialization between Microsoft Detours and Frida-Gum.

5.2.2.1 Suspended-Process Mechanism

The execution sequence for launch mode proceeds through four steps:

1. **Process Instantiation:** The injector starts the process with `CreateProcessA` and the flag `CREATE_SUSPENDED`. When launching a process with this flag, the primary thread is created and suspended before reaching the application entry point [Yos+17].

2. **Hook Activation:** While the main thread remains frozen, the injector applies the Two-Phase Protocol (Section 5.2.1.1):

   - It injects the instrumentation DLL via a remote thread pointing to `LoadLibraryA`.
   - It computes the remote address of `InitializeHooks` and activates all hooks via a second remote thread. In this case, also for Frida there is no risk related to the challenge *Patching a Running Process* presented in Section 3.5.

   Upon completion of `InitializeHooks`, the injector validates the remote thread's exit code: if it is non-zero (indicating hook initialization failure), the injector aborts and terminates the suspended process.

3. **Primary Thread Resumption:** When `InitializeHooks` returns `0` (success), the injector resumes the suspended primary thread, which then runs its startup sequence. Because hooks are already active, any function intercepted by the policy that executes during application startup is captured from its very first call.

4. **Process Termination:** The injector observes instrumented process execution, queries its exit code, and cleanly terminates.

Figure 5.2 compares the dynamic injection pipeline with the suspended launch pipeline, illustrating how both share the Two-Phase Protocol while targeting different process states and lifecycle requirements.

5.2.3 DLL Ejection

When instrumentation is no longer required, the payload must detach cleanly so that the target process can continue running without crashing. As analyzed in Section 2.3.1, restoring the original prologue bytes is necessary but not sufficient: in a multi-threaded application, threads of the target may already be executing within hook functions when detachment begins. If the host unmaps the DLL via `FreeLibrary` while the instruction

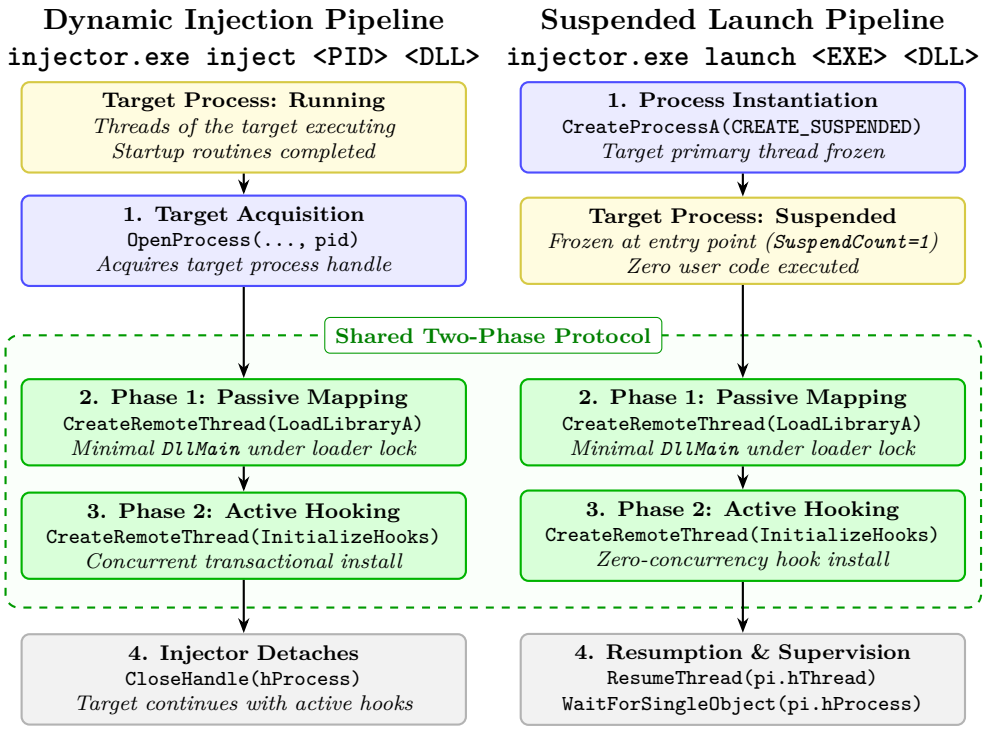


Figure 5.2: Architectural comparison between injection mode and launch mode.

pointer of a thread of the target points to unmapped memory, the process crashes immediately with an Access Violation.

To prevent this, HookDesigner uses a thread-tracking mechanism during hook detachment.

5.2.3.1 Hook Active Thread Tracking

HookDesigner counts the threads that are running inside its hook functions with a RAII helper, `HookActiveThreads`, defined in `_user_logic_helpers.cpp` and shown in Code 5.6.

Code 5.6: HookDesigner implementation of active thread tracking in `_user_logic_helpers.cpp`.

```
static std::atomic<int> activeThreads{0};

// RAII helper to track active hook execution count
struct HookActiveThreads {
    bool active;
    HookActiveThreads() : active(true) {
        activeThreads++;
```

```
    }
    ~HookActiveThreads() {
        if (active) activeThreads--;
    }
    void Release() {
        if (active) {
            activeThreads--;
            active = false;
        }
    }
};
```

- **Atomic Counter:** The counter `activeThreads` is a global atomic variable (`std::atomic<int>`). The constructor of `HookActiveThreads` uses atomic increment (`activeThreads++`) on entry, and its destructor executes atomic decrement (`activeThreads-`) on exit.
- **`Release()`:** lowers the counter before the end of the scope; the flag `active` makes sure that the destructor does not lower it a second time. Only the hook functions of non-returning target functions call it, because in these functions the destructor of the helper never runs (Section 5.1.5.2).

5.2.3.1.1 The `HookActiveThreads` in Each Engine Not every generated hook function declares this RAII helper. As for the re-entrancy guard (Section 5.1.2.1), the choice depends on what the engine already does (Section 3.3.1):

- **Detours.** Detours does not count threads, so every hook function declares it, right after the re-entrancy guard (Code 5.4). Unlike the re-entrancy guard, the helper is not released before the call to the original function: while the original function runs, its return address still points into the hook function, so the thread is still inside the DLL (Section 2.3.1).
- **Frida-Gum, returning functions.** The hook is installed with `gum_interceptor_replace`, and the dispatcher counts the thread from its entry until it returns, the original call included (Section 3.3.1). The hook function does not declare the `HookActiveThreads` helper, and the drain relies on `gum_interceptor_flush()`.
- **Frida-Gum, non-returning functions.** The dispatcher lowers its count only when the function returns, which never happens here: after a call to `ExitThread`, for example, the count stays above zero, and `gum_interceptor_flush()` never reports that the threads have left. For this reason HookDesigner installs these hooks with `gum_interceptor_replace_fast`, which does not use the dispatcher (Section 3.2.1), and the hook function declares the RAII helper, as under Detours.

5.2.3.2 The DLL Ejection Mechanism

When the injector starts ejecting the DLL, it calls the exported function `DeinitializeHooks` in a remote thread through `CallRemoteExport`, as it calls `InitializeHooks` at injection (Section 5.2.1), so that the ejection also runs outside the Loader Lock (Section 4.3.4). The payload then performs the following five-step ejection sequence, illustrated in Algorithm 2:

1. **Terminate Background Listener Threads:** The payload resets the Prometheus HTTP exposer (line 2 of Algorithm 2), so that external HTTP scraping requests do not arrive while metric families and registries are being destroyed.

2. **Revert Hook Patches:** The payload detaches all inline hooks in one transaction of the engine (lines 3–4, Section 3.5). Once the transaction commits, original prologue bytes are restored in memory. New function calls execute the unhooked target code directly; no new threads can enter the hook functions.

3. **Bounded Drain Wait:** Although new calls cannot enter, threads of the target that were already inside the hook functions before Step 2 may still be executing (Section 2.3.1). To synchronize safely, the payload polls a drain condition for at most $T_{\text{drain}} = 2\,\text{s}$ (lines 6–13); Section 5.2.3.3 describes what happens when the limit expires. What "drained" means depends on the engine and, for Frida-Gum, on the policy (Table 5.3); the template fixes this choice once, at code-generation time.

   - Under Detours, every hook function increments the `activeThreads` counter, so the drain condition is that the counter reaches zero.
   - Under Frida-Gum with only returning hooks, the interceptor tracks in-flight invocations itself, and `gum_interceptor_flush()` returning `TRUE` (no thread is still inside the interceptor's trampolines) is sufficient.
   - Under Frida-Gum with at least one non-returning hook, those hooks bypass the interceptor's tracking (Section 5.2.3.1), so both conditions are required.

4. **Destruction of Telemetry Subsystems:** Only after Step 3 confirms that all the threads of the target have drained (*ejected* is TRUE) does the payload destroy the logging and metrics registries (lines 14–15). This ordering is critical: if loggers or registries were destroyed while a thread of the target was still inside `_telemetry_body.cpp` formatting a log or incrementing a counter, the thread would encounter destroyed pointers, triggering a use-after-free crash.

5. **Runtime Deallocation:** Under Frida-Gum, the payload unreferences the interceptor and shuts down the embedded Frida runtime engine (line 16). Finally, `DeinitializeHooks` returns `0` to signal a clean drain and safe detachment to the injector (line 17), or `1` if active threads remain trapped (line 19).

Generated for	Drained Condition
Detours	`activeThreads` $= 0$
Frida-Gum, returning hooks only	`gum_interceptor_flush()`
Frida-Gum, ≥ 1 non-returning hook	`gum_interceptor_flush()` $\wedge$ `activeThreads` $= 0$

Table 5.3: Drain condition for each engine and policy.

Algorithm 2 `DeinitializeHooks`. DRAINED is fixed per engine and policy when the code is generated (Table 5.3); it is not evaluated per call.

Require: $T_{\text{drain}} = 2000\,\text{ms}$, $T_{\text{poll}} = 50\,\text{ms}$
if *hooksAttached* **then**
    stop the metrics exposer (if configured)
    revert all hooks in a single engine transaction
    hooksAttached $\leftarrow$ FALSE
end if
$t_0 \leftarrow$ NOW()
while NOW() $- t_0 < T_{\text{drain}}$ **do**
    if DRAINED **then**
        ejected $\leftarrow$ TRUE
        break
    end if
    sleep T_{poll}
end while
if *ejected* **then**
    destroy the metrics registry and shut down the logger (if configured)
    release the engine runtime (Frida-Gum only)
    return 0
end if
return 1

5.2.3.3 Ejection-Failure DLL Leak Strategy

The bounded wait of Step 3 can expire while a thread is still inside a hook function. This happens when the thread is blocked in the original function (Section 2.3.1), for example in a call to `recv` that waits for network data.

5.2.3.3.1 The Controlled Leak Mechanism

To avoid a crash in this case, HookDesigner implements a defense coordinated between the injected DLL and the injector utility, illustrated in Figure 5.3:

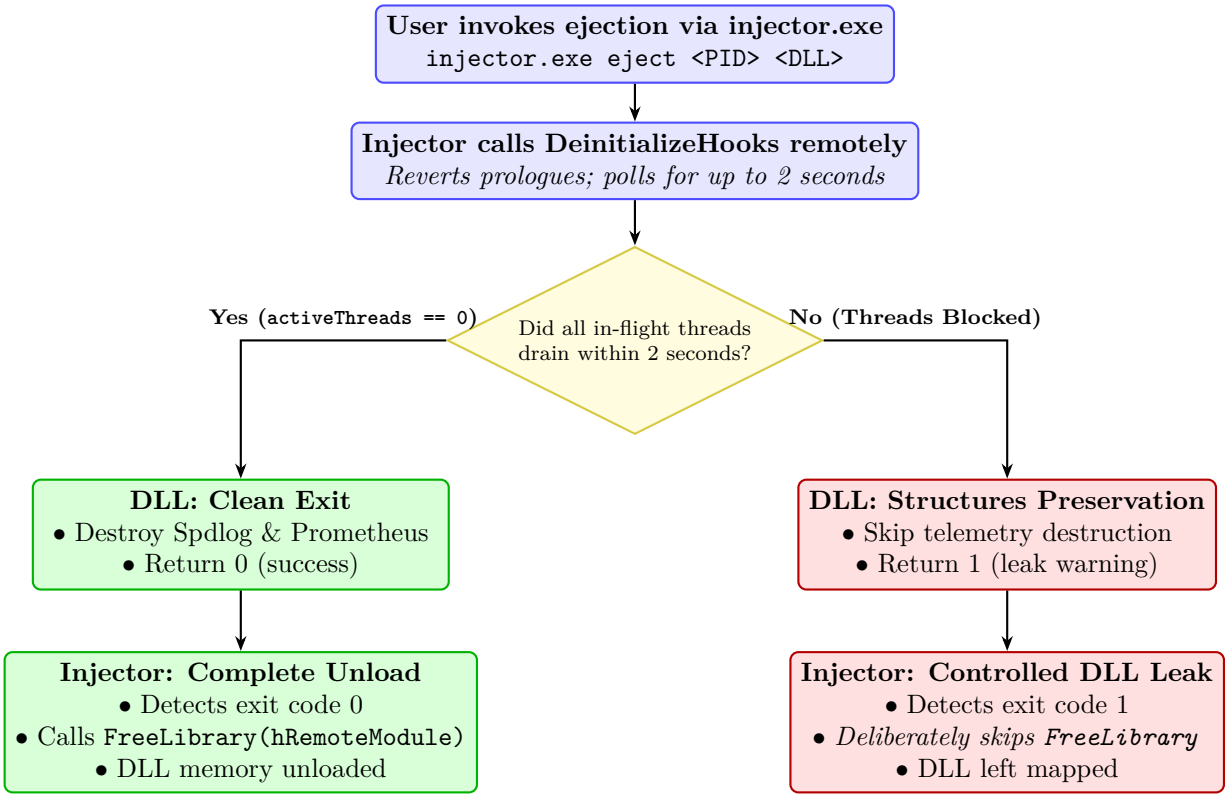


Figure 5.3: Ejection-failure DLL leak strategy.

1. **DLL – Data Structures Preservation (`DeinitializeHooks`):** When the 2-second polling loop expires and threads remain trapped, `DeinitializeHooks` recognizes that clean ejection is impossible. Instead of proceeding with standard cleanup, it:

   - **Preserves Telemetry Registries:** It deliberately skips telemetry data destruction. Leaving these structures intact ensures that when the blocked thread of the target eventually wakes up and runs the post-trampoline code, the telemetry code will not use freed memory.
   - **Returns Status Code:** It returns `1` (thread leak warning code) to the injector, signaling that hooks have been detached but threads remain trapped in hook functions.

2. **Injector – Safety Fallback:** The injector inspects the remote thread's exit code, as shown in Code 5.7:

Code 5.7: Injector fallback logic in `injector.cpp`.

```
DWORD deinitExitCode = (DWORD)-1;

CallRemoteExport(hProcess, dllPath, "DeinitializeHooks", &deinitExitCode);

if (deinitExitCode == 1) { // trapped threads warning: DLL leak fallback
    std::cout << "[WARNING] Hooks successfully detached, but some threads remain
    blocked inside them.\n";
    std::cout << "[WARNING] Skipping FreeLibrary (leaking DLL in target memory) to
     prevent target crash.\n";
} else if (deinitExitCode != 0) { // Detachment error
    std::cerr << "[ERROR] DeinitializeHooks reported error code: 0x"
              << std::hex << deinitExitCode << ". Skipping FreeLibrary.\n";
} else { // Exit code 0: Clean drain confirmed
    std::cout << "Ejecting DLL...\n";
    HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0,
        (LPTHREAD_START_ROUTINE)pFreeLibrary, (LPVOID)hRemoteModule, 0, NULL);
    WaitForSingleObject(hThread, INFINITE);
    CloseHandle(hThread);
}
```

   If `deinitExitCode` is `1`, `injector.exe` outputs a warning and *deliberately skips* calling `FreeLibrary`. If `deinitExitCode` is any other non-zero value, `DeinitializeHooks` failed, for example because one of its exception handlers caught a fault, and `FreeLibrary` is skipped as well. As explained in Section 5.1.4.1, these handlers also skip Steps 4 and 5 of the ejection sequence, so the telemetry structures and, under Frida-Gum, the interceptor are still valid for a thread that may still run the hook code. Only when `deinitExitCode == 0` (clean thread drain confirmed) does the injector safely invoke `FreeLibrary` to unmap the DLL.

The ejection can be repeated later. On a second call, the hooks are already reverted, so `DeinitializeHooks` runs only the drain again; if the blocked threads have left in the meantime, it returns `0`, and the injector unloads the DLL.

From a software engineering perspective, leaking memory is typically considered a defect. However, in DBI—particularly in critical industrial software—leaking a DLL under ejection failure is a safety-oriented design decision.

5.3 Summary

This chapter described how the hook DLL generated by HookDesigner protects the target process while it runs inside it. At transpile time, a type analysis of the parameters chooses safe read helpers for the values that the telemetry captures. A per-thread re-entrancy guard sends the calls that the hook code makes to target functions straight to the original function, and an error guard makes sure that the caller sees the last-error code of the original function, not the one left by the hook code. The DLL is compiled with `/EHa`, so a `catch(...)` block also catches hardware faults. The lifecycle functions undo a failed initialization, so that the hooks are never installed only in part, and each hook function runs the parameter capture, the pre-logic, the post-logic and the telemetry in separate exception boundaries, so that a fault in one of them is contained and the caller still receives a result. The injector loads the DLL with a remote thread, and then calls `InitializeHooks`, in a second remote thread that runs outside the Loader Lock, so that the telemetry threads can start without a deadlock. In launch mode the target is created suspended, so the hooks are active before the program reaches its entry point. At ejection, `DeinitializeHooks` reverts the hooks, waits for the threads that are still inside the hook functions, and destroys the telemetry only after they have left. If some threads stay blocked, the DLL is left loaded on purpose, because unloading it would crash the target, and the ejection can be repeated later.

Appendix A

Code-Cache-Based DBI Frameworks

This appendix extends the summary given in Section 2.4.

A.1 Core Architectural Components

Modern code-cache-based DBI frameworks rely on a unified architectural pattern comprising the following components:

1. **The Dispatcher (Control Flow Manager):** The dispatcher is the central component of the code-cache DBI engine, responsible for orchestrating the execution state. Because the application and the code-cache DBI framework share the same process but operate under different scopes, they must safely switch between "framework mode" and "application mode". The dispatcher triggers a context-switch mechanism that carefully saves and restores the application's state whenever execution exits or enters the code cache.

2. **JIT Compiler:** Instead of translating the entire binary before execution, these architectures translate the application's instructions dynamically in small fragments. During this translation phase, the framework decodes and encodes (into the framework's abstraction) the original instructions and has the opportunity to insert custom instrumentation code or patches before emitting the new sequence.

3. **The Code Cache (The Execution Sandbox):** To mitigate the performance penalty of repeatedly translating the same code, code-cache-based DBI frameworks employ a software-based code cache. The translated and potentially instrumented fragments are stored in this sandboxed memory region. When the application

re-executes a previously translated section of code, it runs directly from the code cache, which significantly optimizes execution speed.

4. **The Client Interface (Instrumentation API):** This is the abstraction layer exposed to the developer. It provides the necessary functions to inspect the program state, traverse instruction sequences, and specify exactly where and how instrumentation should be injected.

A.2 The Execution Lifecycle: The JIT Loop

Figure A.1 shows how these components interact at runtime, through the execution lifecycle of a standard block of code:

1. *Interception:* The target application attempts to execute a block of original code. The code-cache DBI engine intercepts this attempt and yields control to the dispatcher.

2. *Cache Lookup:* The dispatcher queries a fast-lookup hash table to check if the target memory address has already been translated. If it results in a cache hit, the CPU is redirected to the code cache.

3. *Translation (On Cache Miss):* If the code has not been encountered before, the JIT compiler decodes original binary instructions one by one to form a straight-line sequence that terminates at a control transfer, such as a branch, call, or return.

4. *Instrumentation:* The engine queries the Client Interface to determine if the user wishes to instrument the block or a single instruction within it. If so, the instrumentation instructions are injected.

5. *Emission and Linking:* Most code-cache-based DBI frameworks use this phase to apply patches to the control-flow instructions at the end of the sequence, guaranteeing that when the block finishes running, it will redirect back to the framework rather than jumping to a native memory address. If the block ends in a direct branch to another previously translated block, the engine "links" them together directly in the cache (if it was an indirect branch, it cannot know the target before executing it), allowing the CPU to bypass the dispatcher in future iterations. The modified block is emitted into the code cache.

6. *Execution:* The dispatcher performs a context switch, restoring the application state, and commands the CPU to execute the newly cached block natively.

7. *Code Cache Exit:* Execution remains safely inside the code cache—often jumping directly from one translated block to another through "linking"—until it hits a boundary. This boundary might be an unlinked branch or an indirect branch whose target has not been translated yet. When this happens, the block executes an exit stub which saves the CPU's state and transfers control back to the dispatcher, restarting the loop.

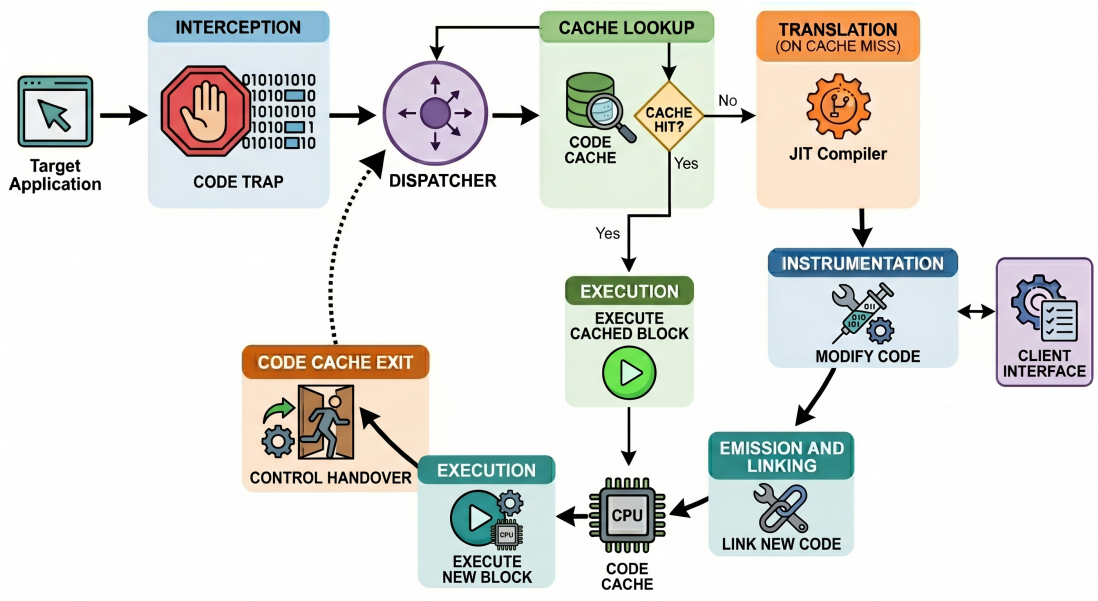


Figure A.1: The execution lifecycle of a code-cache-based DBI framework.

A.3 Comparison with Inline Hooking

Table A.1 summarizes the differences between the two DBI paradigms: inline hooking (Section 2.3) and the code-cache-based DBI described in this appendix.

A.4 The Client Interface: Instrumentation vs. Analysis Functions

Code-cache-based DBI frameworks require the developer to interact with the engine using a *two-tiered callback paradigm*. The developer must supply two distinct types of functions to the engine, each operating at fundamentally different times during the execution lifecycle:

Dimension	Inline hooking	Code-cache-based DBI
Mechanism	A jump written at the start of the target function sends each call to the hook function; the trampoline runs the moved instructions and jumps back to the rest of the function.	A dispatcher passes each new block of code to a JIT compiler, which translates it and inserts the instrumentation code; the CPU runs only the translated copies stored in the code cache.
Granularity	Function level: the hook function runs when a target function is called, and can also run code after the original function returns.	Instruction level: instrumentation code can be inserted at any instruction, for example to log every conditional branch.
Runtime overhead	Lower: only the calls to target functions pay an extra cost (jump, hook function and trampoline); the rest of the program runs as before.	Higher: every block of code that the program runs is translated before it runs, even when it is not instrumented, and the dispatcher runs again when a translated block cannot jump directly to the next one.
Machine state	Preserved by the compiler, because the hook function is an ordinary function that follows the ABI; the hook function must also declare exactly the parameter list and return type of the target.	Saved and restored by the engine, because engine-generated code runs between the program and the instrumentation code.
Original code	Changed in memory: the first bytes of each target function are overwritten, and written back when the hooks are removed.	Never changed, and never executed directly by the CPU.
Engine complexity	Lower: the engine changes only the prologue of each target function, so it must find where the prologue instructions end and write the jump safely while other threads run.	Higher: the engine needs a dispatcher and a JIT compiler that can translate every instruction that the program runs.

Table A.1: Comparison of the two main DBI paradigms.

- *The Instrumentation Function (The "Where" and "What"):* This callback is invoked by the code-cache DBI engine *only once* per newly discovered sequence of instructions during the JIT compilation phase. Its purpose is to inspect the static properties of the instructions (e.g., opcodes, operands) and decide *if* and *where* custom code should be injected. If injection is required, it uses the API to queue the insertion of the analysis function.
- *The Analysis Function (The "Action"):* This is the actual custom payload injected into the code cache. Unlike the instrumentation function (executed only at translation time), it is executed *every time* the CPU runs the corresponding instrumented block of code. For this reason, the analysis function must be heavily optimized and efficient.

A.5 High-Level Abstraction API: Intel Pin

The level of abstraction provided by the Client Interface significantly alters how developers interact with the engine. Intel Pin [Luk+05] is a standard academic code-cache-based DBI framework with a proprietary (free for non-commercial use) license. It provides a heavily abstracted C++ API.

To utilize Intel Pin, developers do not modify the core engine itself. Instead, they write a custom C++ program known as a *PINtool*. A PINtool represents the physical implementation of the Client Interface discussed previously; it is the user-defined module that contains both the Instrumentation Functions and the Analysis Functions. Architecturally, a PINtool is compiled as a standard shared library (a DLL on Windows). When the user launches a target application under Pin, the engine automatically injects the PINtool DLL into the target process's memory space. Once loaded, the engine invokes the PINtool's `main` function, allowing the tool to register its specific callbacks. Code A.1 shows the granularity of code-cache-based DBI: the tool logs the conditional branches, to find which paths the program takes.

Code A.1: An example of a PINtool for branch logging.

```
// Analysis function (Runs at execution time in the Code Cache)
void record_branch(ADDRINT instruction_address, ADDRINT target_address, BOOL taken) {

  if (taken) { /* Log Taken */ }
  else       { /* Log Not Taken */ }
}

// Instrumentation function (Runs at JIT compilation time)
void trace(INS instruction, void *v) {
```

```
  // Pin abstracts away module resolution: instrument
  // only target application instructions, filter possible external libraries
  RTN rtn = INS_Rtn(instruction);
  if (RTN_Valid(rtn) && IMG_IsMainExecutable(SEC_Img(RTN_Sec(rtn)))) {
    // If it is inside the target executable, check if it is an IF branch
    if (INS_IsBranch(instruction) && INS_HasFallThrough(instruction)) {
      // Inject analysis function right before the jump happens.
      INS_InsertCall(instruction, IPOINT_BEFORE, (AFUNPTR)record_branch,
      IARG_INST_PTR,
      IARG_BRANCH_TARGET_ADDR,
      IARG_BRANCH_TAKEN,
      IARG_END);
    }
  }
}
```

A.6 Low-Level API: DynamoRIO

DynamoRIO [BGA03] is the open-source alternative that exposes a lower-level C API. The equivalent branch-logging example seen previously for Pin is implemented in Code A.2.

Code A.2: DynamoRIO branch instrumentation example.

```
// Analysis function
static void record_branch(app_pc instr_addr, app_pc target_addr, int taken) {

  if (taken) { /* Log Taken */ }
  else       { /* Log Not Taken */ }
}

// Instrumentation function
static dr_emit_flags_t event_bb_analysis(void *drcontext, void *tag, instrlist_t *bb,
   instr_t *inst, bool for_trace, bool translating,
void *user_data) {

  // Manual module tracking to filter out OS libraries
  app_pc instr_addr = instr_get_app_pc(inst);
  module_data_t *mod = dr_lookup_module(instr_addr);
  bool is_main_exec = false;

  if (mod != NULL) {
    module_data_t *main_mod = dr_get_main_module();
    if (main_mod != NULL) {
      if (mod->start == main_mod->start) { is_main_exec = true;
      }
      dr_free_module_data(main_mod);
```

```
    }
    dr_free_module_data(mod);
  }

  if (!is_main_exec) return DR_EMIT_DEFAULT;
  // Check if it is a conditional branch (CBR)
  if (instr_is_cbr(inst)) {
    // Inject analysis function into the basic block (bb)
    dr_insert_cbr_instrumentation(drcontext, bb, inst, (void *)record_branch);
  }
  return DR_EMIT_DEFAULT;
}
```

Using DynamoRIO, developers can directly manipulate the instruction flow. In Pin, developers generally "wrap" existing instructions (adding a call before or after an instruction) but cannot easily modify the instruction itself. In DynamoRIO, the developer is handed a linked list of instructions (`instrlist_t`) and can easily manipulate or modify them (e.g., reordering them or moving a load instruction earlier to test for race condition).

Furthermore, because Pin is designed to be highly opaque, using `INS_InsertCall` prompts Pin's JIT compiler to automatically analyze the custom analysis function for deciding which registers need to be saved and restored based on what the C++ code actually uses. With DynamoRIO, developers can manage the state and flags manually for specialized analysis.

In conclusion, while DynamoRIO may present a steeper learning curve than Intel Pin, it affords developers greater transparency, providing fine-grained control over the translated code and the code cache.

A.7 Summary of Code-Cache-Based DBI Frameworks

To summarize the key differences between the analyzed code-cache-based DBI engines, Table A.2 compares the features and capabilities of Intel Pin and DynamoRIO.

Feature	Intel Pin	DynamoRIO
API Style	High-level C++ API	Low-level C API
Code Modification	High-level wrapping (instrumentation only before/after instructions)	Direct instruction manipulation
State/Flags Handling	Fully automatic (always analyzed and optimized by JIT compiler)	Manual or automatic (developer can choose)
Platform Support	Windows, Linux	Windows, Linux, macOS, Android (x86, x64, ARM, ARM64)
License	Proprietary (Free for non-commercial use)	BSD (Open Source)

Table A.2: Comparison of the analyzed code-cache-based DBI frameworks.

Appendix B

How a Program Opens a File on Windows

The running example of this thesis (Section 1.2) stops a program from opening some files. Its hooks intercept two functions of the Windows *native API*, `NtCreateFile` and `NtOpenFile`. This appendix explains what these functions are, why the example hooks them instead of a better-known function such as `CreateFileW`, and the types of their parameters and of their result that the listings of this thesis use.

B.1 The Layers of a File Open

A program rarely talks to the operating system kernel directly. It calls functions of the Win32 API, such as `CreateFileW`, or functions of a library built on top of it, such as `_wfopen` of the C runtime or the `std::ifstream` class of the C++ standard library. The Win32 functions in turn call the functions of the native API, which are exported by `ntdll.dll`; these functions are the last step in user mode, and they enter the kernel through a system call [Yos+17].

For files, the two native functions are:

- `NtCreateFile`, which "creates a new file or directory, or opens an existing file, device, directory, or volume" [Mic26k];
- `NtOpenFile`, which "opens an existing file, device, directory, or volume, and returns a handle for the file object" [Mic26l].

This layering decides where a hook must be placed. A hook on `CreateFileW` sees only the

programs that call `CreateFileW`: a program that calls `NtOpenFile` directly passes below it. A hook on `NtCreateFile` and `NtOpenFile`, instead, sees the file opens of every layer above them, whatever function the program used. For this reason the running example hooks the two native functions. A program can still skip `ntdll.dll` by executing the system call instruction itself; no user-mode hook can see such a call.

B.2 Parameters and Result

Code B.1 shows the declaration of `NtOpenFile`, the function used by most listings of this thesis. `NtCreateFile` has the same first four parameters, followed by seven more that describe how the file is created [Mic26k].

Code B.1: Declaration of `NtOpenFile` [Mic26l].

```
NTSTATUS NTAPI NtOpenFile(
    PHANDLE            FileHandle,       // out: the handle of the opened file
    ACCESS_MASK        DesiredAccess,    // in:  the rights requested
    POBJECT_ATTRIBUTES ObjectAttributes, // in:  the name of the file
    PIO_STATUS_BLOCK   IoStatusBlock,    // out: details on the result
    ULONG              ShareAccess,      // in:  what other opens may do
    ULONG              OpenOptions);     // in:  options of the open
```

`DesiredAccess` is a bit mask of the rights that the caller asks for, for example `FILE_READ_DATA` to read the data of the file and `FILE_WRITE_DATA` to write it [Mic26k].

`ObjectAttributes` is a pointer to an `OBJECT_ATTRIBUTES` structure, which holds the name of the file in two members [Mic26m]:

- `ObjectName` points to a `UNICODE_STRING`, a string with an explicit length that is not guaranteed to end with a NUL character [Mic26q];
- `RootDirectory` is an optional handle to a directory. If it is `NULL`, `ObjectName` must contain the full path of the file, for example `\??\C:\data\a.txt`; otherwise, `ObjectName` is a name relative to that directory, for example `a.txt` [Mic26k; Mic26m].

So the full path of a file is not always in the name that a hook function receives: for a relative open, part of it is hidden behind the `RootDirectory` handle.

`NTSTATUS`: Native functions do not use `GetLastError`: they return a 32-bit `NTSTATUS` value. Since `NTSTATUS` is a signed type, a negative value is a warning or an error. The running example uses the error `STATUS_ACCESS_DENIED` (`0xC0000022`): "a process has requested access to an object but has not been granted those access rights" [Mic26a].

Appendix C

Preliminary Experiments

To empirically validate the concepts discussed in Chapters 2 and 3 and to fully understand the operational constraints, security boundaries, and low-level deployment challenges of each framework, I developed a preliminary suite of testing applications. I made all the source code and the custom injection utilities available in the project's repository.[1]

I structured the experimental activity into three different scenarios, each designed to evaluate specific architectural properties of the instrumentation engines.

C.1 Scenario 1: Single-Threaded Hooking and Process Injection/Ejection Mechanics

- **Test Rationale:** This preliminary scenario aimed to verify the fundamental lifecycle management of DBI within the Windows ecosystem.

  This involved implementing a custom, dual-mode utility (`injector.cpp`) to handle both the insertion and the ejection of the instrumentation payload.

  Concurrently, the test evaluated the syntactic and structural paradigms of the three inline hooking frameworks (Microsoft Detours, MinHook, and Frida-Gum) when intercepting a standard system API—`MessageBoxW` in `user32.dll`.

- **Empirical Results:** Every analyzed framework successfully intercepted the execution flow in the pre-logic. This capability allowed for the modification of string pointers targeting the dialog box text and caption, the reassignment of the configuration flags to `MB_YESNO`, and the runtime mutation of the return value in the post-logic,

[1] https://github.com/davde342/MasterThesis/tree/main/code/first_examples

effectively forcing an `IDYES` outcome regardless of the user's explicit interaction.

Furthermore, forcing the programmatic module ejection successfully triggered the `DLL_PROCESS_DETACH` branch within each hook's entry point, safely restoring the original prologue instruction bytes and confirming a correct ejection in single-threaded environments.

- **Encountered Obstacles:** In this idealized single-threaded scenario, no major problems occurred. Setting up Microsoft Detours was straightforward due to its thoroughly documented APIs, the abundance of reference examples, and a standard paradigm that is conceptually simpler than that of Frida-Gum.

  MinHook demonstrated a similar workflow to Detours but simplified the implementation further by automatically managing transactional patching internally.

  Frida-Gum was harder to learn: its architecture is more complex, its model is different, and its documentation is less complete. To understand its API, I had to read its source code in the official Frida repository.

C.2 Scenario 2: Multi-Threaded Hooking and Concurrency Management

- **Test Rationale:** To simulate a realistic production workload and test the instrumentation frameworks under thread concurrency, I used a multi-threaded target program (`target_multithread.cpp`). This target utilizes a synchronized task queue where a producer thread continuously queues activities with varying workloads, while three concurrent consumer threads process these tasks in parallel by invoking the exported routines `ProcessTask` and `LogTaskResult`.

- **Empirical Results:** Real-time monitoring of thread behaviors was achieved, allowing a dynamic optimization policy: the injected hook intercepted high-latency tasks (>500ms) and programmatically constrained their complexity metrics to 100ms prior to native routing, illustrating the possibility of inline hooking for runtime policy enforcement and load adaptation.

- **Encountered Obstacles:** I faced two major obstacles in this scenario. First, performing hook detachment in a concurrent context introduced a possible race condition: threads of the target could still be inside the hook function when the library was unmapped, which caused an immediate Access Violation (`0xC0000005`).

  Second, in Frida-Gum, intercepting routines that process floating-point arguments (such as the `double` parameter in `LogTaskResult`) failed, because these parameters are passed in the `XMM0`–`XMM3` registers under the Windows x64 ABI, while `gum_`

`invocation_context_get_nth_argument` reads only the general-purpose registers (Section 3.2).

- **Implemented Mitigations:** To address the detachment race condition in concurrent environments, I used framework-specific synchronization (the problem is described in Section 2.3.1, and the support of each engine in Section 3.3.1).

  As detailed in Section 3.2, to bypass the floating-point register limitation inherent in Frida-Gum's Invocation Listener mode, the high-level callback attachment was discarded in favor of Frida-Gum's Function Replacement mode via `gum_interceptor_replace`, allowing the C/C++ compiler to handle exact register allocation natively.

C.3 Scenario 3: Instruction-Level Assembly Tracking via Code-Cache-Based DBI Engines

- **Test Rationale:** To achieve low-level instruction granularity, a target program containing conditional branches was compiled (`PIN_dynamo_target.cpp`), alongside an Intel Pin tool (`pintool.cpp`) and a DynamoRIO client (`dynamo.c`). The rationale was to program the underlying JIT compilers to isolate all conditional branch instructions (`INS_IsBranch` / `instr_is_cbr`) and dynamically inject an analysis callback (`record_branch`) used to log the target address (taken branch). The APIs of both engines are described in Appendix A.

- **Empirical Results:** Both code-cache-based DBI engines produced logs of the taken branches. Using Intel Pin, it was also possible to write the source line number corresponding to each taken branch.

- **Encountered Obstacles:** An initial operational challenge involved navigating the different APIs of the two frameworks; DynamoRIO exposes lower-level utilities compared to the highly abstracted API of Intel Pin, resulting in a steeper learning curve. Beyond API familiarity, the primary technical obstacle across both engines was attempting to log not only the target runtime address of each taken branch but also its source code line number from `PIN_dynamo_target.cpp` (compiled with debug symbols) to maximize execution trace readability.

- **Implemented Mitigations:** I found that Intel Pin needs `PIN_GetSourceLocation` for this. However, direct invocation of this routine inside the analysis function triggers a critical runtime error due to missing client locks. To mitigate this, calls to `PIN_GetSourceLocation` were wrapped inside a thread-safe synchronization pair using `PIN_LockClient()` and `PIN_UnlockClient()`, ensuring proper access to Pin's symbol database.

Bibliography

[BGA03] Derek Bruening, Timothy Garnett, and Saman P. Amarasinghe. "An Infrastructure for Adaptive Dynamic Optimization". In: *1st International Symposium on Code Generation and Optimization (CGO 2003), 23-26 March 2003, San Francisco, CA, USA*. IEEE Computer Society, 2003, pp. 265–275. DOI: 10.1109/CGO.2003.1191551. URL: https://doi.org/10.1109/CGO.2003.1191551.

[Che16] Raymond Chen. *If relocated DLLs cannot share pages, then doesn't ASLR cause all pages to be non-shared?* The Old New Thing, Microsoft Developer Blogs. Apr. 13, 2016. URL: https://devblogs.microsoft.com/oldnewthing/20160413-00/?p=93301.

[Fri26] Frida Developers. *Frida-Gum: Cross-platform instrumentation and introspection library*. Version 17.9.3. 2026. URL: https://github.com/frida/frida-gum.

[HB99] Galen C. Hunt and Doug Brubacher. "Detours: Binary Interception of Win32 Functions". In: *Proceedings of the 3rd USENIX Windows NT Symposium, Seattle, WA, USA, July 12-13, 1999*. USENIX Association, 1999, pp. 135–143. URL: https://www.cs.columbia.edu/~junfeng/10fa-e6998/papers/detours.pdf.

[Kag26] Tsuda Kageyu. *MinHook: The Minimalistic x86/x64 API Hooking Library for Windows*. Version 1.3.4. 2026. URL: https://github.com/TsudaKageyu/minhook.

[Lev00] John R. Levine. *Linkers and Loaders*. San Francisco, CA: Morgan Kaufmann, 2000. ISBN: 1-55860-496-0.

[LLV26] LLVM Developer Group. *Clang: A C Language Family Frontend for LLVM*. Version 18.1.1. 2026. URL: https://clang.llvm.org/.

[Luk+05] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. "Pin: building customized program analysis tools with dynamic instrumentation". In: *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2005, Chicago, IL, USA, June 12-15,*

2005. Ed. by Vivek Sarkar and Mary W. Hall. ACM, 2005, pp. 190–200. DOI: 10.1145/1064978.1065034. URL: https://doi.org/10.1145/1064978.1065034.

[Mel26] Gabi Melman. *spdlog: Very fast, header-only/compiled, C++ logging library*. Version 1.17.0. 2026. URL: https://github.com/gabime/spdlog.

[Mic26a] Microsoft Corporation. *[MS-ERREF]: NTSTATUS Values*. 2026. URL: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-erref/596a1078-e883-4972-9bbc-49e60bebca55.

[Mic26b] Microsoft Corporation. */DYNAMICBASE*. 2026. URL: https://learn.microsoft.com/en-us/cpp/build/reference/dynamicbase.

[Mic26c] Microsoft Corporation. */EH (Exception Handling Model)*. 2026. URL: https://learn.microsoft.com/en-us/cpp/build/reference/eh-exception-handling-model.

[Mic26d] Microsoft Corporation. *Argument Passing and Naming Conventions*. 2026. URL: https://learn.microsoft.com/en-us/cpp/cpp/argument-passing-and-naming-conventions.

[Mic26e] Microsoft Corporation. *Detours: Software package*. Version 4.0.1. 2026. URL: https://github.com/microsoft/Detours.

[Mic26f] Microsoft Corporation. *Dynamic-Link Library Best Practices*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/dlls/dynamic-link-library-best-practices.

[Mic26g] Microsoft Corporation. *GetFinalPathNameByHandleW function (fileapi.h)*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-getfinalpathnamebyhandlew.

[Mic26h] Microsoft Corporation. *GetLastError function (errhandlingapi.h)*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-getlasterror.

[Mic26i] Microsoft Corporation. *GetModuleHandleA function (libloaderapi.h)*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-getmodulehandlea.

[Mic26j] Microsoft Corporation. *GetProcAddress function (libloaderapi.h)*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-getprocaddress.

[Mic26k] Microsoft Corporation. *NtCreateFile function (winternl.h)*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntcreatefile.

[Mic26l] Microsoft Corporation. *NtOpenFile function (winternl.h)*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntopenfile.

[Mic26m] Microsoft Corporation. *OBJECT_ATTRIBUTES structure (ntdef.h)*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/api/ntdef/ns-ntdef-_object_attributes.

[Mic26n] Microsoft Corporation. *OpenProcess function (processthreadsapi.h)*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-openprocess.

[Mic26o] Microsoft Corporation. *PE Format*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format.

[Mic26p] Microsoft Corporation. *Storage classes (C++)*. 2026. URL: https://learn.microsoft.com/en-us/cpp/cpp/storage-classes-cpp.

[Mic26q] Microsoft Corporation. *UNICODE_STRING structure (ntdef.h)*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/api/ntdef/ns-ntdef-_unicode_string.

[Mic26r] Microsoft Corporation. *Windows Data Types*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/winprog/windows-data-types.

[Mic26s] Microsoft Corporation. *x64 ABI Conventions*. 2026. URL: https://learn.microsoft.com/en-us/cpp/build/x64-software-conventions.

[Mp26] Jupp Müller and prometheus-cpp Contributors. *prometheus-cpp: Prometheus Client Library for Modern C++*. Version 1.3.0. 2026. URL: https://github.com/jupp0r/prometheus-cpp.

[Pal24] Pallets Projects. *Jinja: A full-featured template engine for Python*. Version 3.1.6. 2024. URL: https://jinja.palletsprojects.com/.

[The26a] The MITRE Corporation. *Process Injection: Dynamic-link Library Injection, Sub-technique T1055.001*. 2026. URL: https://attack.mitre.org/techniques/T1055/001/.

[The26b] The Prometheus Authors. *Prometheus: An open-source systems monitoring and alerting toolkit*. 2026. URL: https://prometheus.io/.

[Tho20] Jacob Thompson. *Six Facts about Address Space Layout Randomization on Windows*. Mandiant. Mar. 17, 2020. URL: https://cloud.google.com/blog/topics/threat-intelligence/six-facts-about-address-space-layout-randomization-on-windows.

[Yos+17] Pavel Yosifovich, Alex Ionescu, Mark E. Russinovich, and David A. Solomon. *Windows Internals, Part 1: System architecture, processes, threads, memory management, and more*. 7th. Microsoft Press, 2017. ISBN: 978-0-13-398634-1.

[Zf26] Victor Zverovich and fmt Contributors. *fmt: A modern formatting library*. Version 12.1.0. 2026. URL: https://github.com/fmtlib/fmt.